

Job Autopilot

From relevant vacancy
to verified submission.

A complete implementation contract for a local-first, deployable job-application system:
truthful tailoring, free-model orchestration, reliable browser execution, and
evidence-backed recovery.

18

IMPLEMENTATION PHASES

3

DEPLOYMENT PROFILES

32

INTEGRATED TEST SCENARIOS

Prepared for Mehul | Netherlands job search

16 September 2026

Design specification, not an implemented or benchmarked product.
Includes a private candidate-policy section; sanitize before public release.

Contents

Read chapters 01-12 for the system contract, then execute phases P00-P17. The appendices contain API examples, tests, runbooks, coding instructions, and sources. Contents, bookmarks, source references, and external citations are clickable.

01. Executive contract and how to use this specification

02. Product goals, operating limits, and release measures

03. Career Ops reference review and reuse strategy

04. Candidate truth, Netherlands eligibility, and targeting

05. Reference architecture and repository boundaries

06. Durable data model, queue, and state transitions

07. Job discovery and fit pipeline

08. Free-model gateway and bounded agent orchestration

09. Document factory and application packets

10. Browser adapter SDK, accounts, and exception handling

11. Final submission protocol and reconciliation

12. Security, deployment, operations, and public release

P00. Repository audit and implementation charter

P01. Runnable skeleton, developer experience, and CI

P02. Persistence, durable work, and audit events

P03. Candidate evidence and standing authorization

P04. Discovery, job identity, and historical imports

P05. Free-only inference and evaluated matching

P06. Tailored CVs, letters, and answer packets

P07. Mock ATS and browser filling contract

P08. Actual submission vertical slice

4

6

8

10

12

14

16

18

20

22

24

26

29

30

31

32

33

34

35

36

37

P09. High-yield portal coverage 38

P10. Accounts, secure handoffs, and exception resolution 39

P11. Email integration and receipt reconciliation 40

P12. Bounded multi-agent orchestration and cost optimization 41

P13. Unattended Windows autopilot and throughput 42

P14. Fully deployable server and hybrid profiles 43

P15. Security, fault injection, restore, and release gates 44

P16. Outcomes, calibration, and continuous improvement 45

P17. Release, portfolio, and autonomous maintenance handoff 46

Appendix A. Interface contracts and configuration examples 48

Appendix B. Test matrix and definition of done 50

Appendix C. Operator runbooks 52

Appendix D. Autonomous coding execution protocol 54

Appendix E. Priorities, tradeoffs, and future extensions 56

Appendix F. Sources and verification notes 57

01. Executive contract and how to use this specification

Version 2.0 | 16 September 2026 | Implementation specification, not an implemented product

Purpose. Build a serious, local-first job-search and application system that discovers relevant vacancies, produces truthful tailored documents, completes company application forms, makes authorized final submissions, and records verifiable outcomes. It must operate unattended between genuine exceptions, recover after interruptions, and also be deployable on a server. A polished dashboard is useful; actual submitted applications are the product's central outcome.

This edition is a coherent replacement specification for the earlier planning drafts, not an additional architecture to combine with them. It selects a database-backed durable workflow as the initial runtime. Temporal, Redis, Kubernetes, a vector database, and unrestricted swarms are not mandatory infrastructure. They can become justified later decisions, but must not delay the first working submission path.

The delivery promise

- Complete an end-to-end workflow: discover, normalize, deduplicate, assess, prepare, fill, submit, reconcile, and report.
- Include a real auto-submit mode under standing owner authorization. Do not quietly deliver only a cover-letter generator, application tracker, browser macro, or human-final-click product.
- Support a native Windows installation, a fully server-hosted installation, and a hybrid arrangement with a local browser worker.
- Use OpenRouter free models as the default external inference path, with a hard prohibition on paid fallback unless the owner changes a separate budget policy.
- Keep a durable exception queue. One CAPTCHA, unknown declaration, unavailable model, or unsupported employer portal must not stop unrelated work.
- Make reliability, truthful claims, observable evidence, and measured evaluation the project's portfolio differentiators. Describe it as advanced only when the published measurements support that description.

Scope of autonomy

The owner can authorize the system once to submit applications meeting a versioned policy, create ordinary applicant accounts using verified details, and reuse approved answers. Routine eligible applications then require no fresh confirmation. The app must display what this authorization covers and allow immediate revocation. A chat instruction is useful context but is not a substitute for recording the application's own durable policy and user identity during onboarding.

CAPTCHA handling is detection, preserved progress, and a secure human handoff or a portal-supported authorized alternative. This specification does not include CAPTCHA solving services, challenge-token reuse, fingerprint deception, proxy rotation to evade blocks, or anti-bot bypass. The worker moves to other vacancies while an exception waits. An unsupported or prohibited automated application path remains unsupported; being publicly accessible is not sufficient authorization for every use.

Non-negotiable definitions

Term	Exact meaning
Prepared	Documents and proposed answers exist. Nothing has necessarily been sent.
Filled	Required fields and uploads have been validated in the current form.
Attempted	A final submission was initiated; outcome may be unknown.
Confirmed submission	Job-specific portal acknowledgment, account record, or correlated receipt establishes receipt of the application.
Historical submitted	The owner says it was submitted, but the new system may have no independent receipt.
Needs help	A precise unresolved item is saved with a resumable state and a reason.
Match score	A transparent prioritization score, not an interview or selection probability.
Production ready	The specified release gates passed with evidence; it does not mean every website is supported.

Reading and implementation order

Read chapters 01-12 before coding. They define the invariants and interfaces. Execute phases P00-P17 in order unless a phase explicitly allows independent work. Each phase supplies inputs, work packages, acceptance gates, evidence, and a resumption handoff. The appendices provide test cases, operational procedures, implementation instructions, and sources.

The Markdown companion is the editable source of truth for this document. Record subsequent changes as architecture decisions and versioned amendments. Do not rewrite requirements to make incomplete functionality appear complete. The project has three useful delivery points: a controlled live-submission vertical slice at P08, a daily local autopilot at P13, and a deployable, release-tested portfolio product at P17.

02. Product goals, operating limits, and release measures

What success means

The primary daily counter is **unique, appropriate, confirmed applications**, not links collected, drafts generated, buttons clicked, or browser sessions opened. The operating target is 50 confirmed applications per day when sufficient suitable vacancies, portal access, model capacity, and account readiness exist. Fifty is a target, not a guarantee or a reason to ignore eligibility, repeat applications, or weaken factual checks.

Early release uses a controlled ramp: prove one live route, then a small canary group, then 20, then the requested 50-per-day target after the evidence supports it. The owner can configure the daily target; the system must show the limiting stage instead of silently reducing it. A shortage of eligible vacancies is a product state, not a worker failure. Continuing 24/7 means watching legitimate sources and processing new work, not inventing endless fresh opportunities.

Requirements catalogue

ID	Mandatory behavior	Main evidence
R01	Ingest direct company vacancies and supported job feeds	Connector fixtures and live read-only samples
R02	Avoid duplicate applications across sources and restarts	Identity tests and crash suite
R03	Rank Netherlands-compatible roles with explicit reasons	Labeled evaluation set
R04	Preserve verified candidate facts and original files	Provenance and artifact manifests
R05	Generate a tailored CV and optional motivation letter	Claim audit, PDF text and render tests
R06	Fill and finally submit supported forms	Mock receipts plus a genuine controlled live receipt
R07	Recover ambiguous submissions without blind replay	Fault injection and reconciliation evidence
R08	Continue around challenges and missing information	Independent queue progress tests
R09	Prevent paid inference in free-only mode	Pricing/firewall tests and usage ledger
R10	Run without Codex remaining open	Restart and unattended soak tests
R11	Deploy in local, server, and hybrid profiles	Installation and restore demonstrations
R12	Publish a reproducible, sanitized GitHub project	Clean clone CI and public-data audit

Metrics with honest denominators

Track discovered unique jobs, live jobs, policy-eligible jobs, prepared packets, forms attempted, confirmed submissions, ambiguous attempts, exceptions, and later outcomes. Report automation completion as confirmed / eligible supported attempts, excluding duplicates but not hiding failures. Separately report coverage as supported eligible jobs / all eligible jobs. A high completion rate on one easy portal is not broad internet coverage.

Report time-to-first-seen, first-seen-to-triage, triage-to-ready, ready-to-receipt, model calls per packet, correction rate, unsupported-form rate, account friction, and evidence confidence. Segment by connector, adapter version, role

family, seniority, language gate, sponsorship signal, and date cohort. Never put candidate names or emails into metric labels.

The first release gates are absolute correctness gates: no unsupported candidate claims in the gold set; no blind duplicate in crash fixtures; every confirmed state has evidence; zero paid requests in free-only tests. Initial operational targets are at least 95% completion on supported mock forms, an exception surfaced within one minute of detection, and process recovery within two minutes in a controlled local test. These are design targets to measure, not results already achieved.

Capacity planning

Use measured stage times rather than promising universal speed. For example, two browser workers spending six minutes per supported application have a theoretical ceiling of 20 per hour before accounts, challenges, throttling, document generation, and supply reduce it. This is arithmetic, not a benchmark. One worker per employer domain is the initial default; total browser concurrency starts at two and is tuned against memory and portal feedback.

If 50 submitted jobs require 80 eligible opportunities and only 25 appear today, no orchestration framework closes the gap. Show the supply deficit and broaden approved role aliases or company coverage, not qualifications. If models are unavailable, continue collection, deduplication, approved deterministic answers, cached packets, and reconciliation. Defer only work that genuinely needs new inference.

Explicitly outside the first production scope

The first release is single-owner software, not a public multi-tenant recruiting SaaS. It does not promise automatic completion of hiring tests, recruiter conversations, employment contracts, immigration filings, payments, or background-check declarations. It does not scrape logged-in LinkedIn pages against restrictions, purchase services, or claim it can access every website. These boundaries keep the actual submission feature achievable and testable.

03. Career Ops reference review and reuse strategy

What was inspected

The reference is `career-ops-hq/career-ops`, reviewed at commit `aac998c7ed7248ea853b720ceeb1fdbeb322fc5d` (version 1.32.0). Pin this revision in the research note; main can change. Review was read-only. Upstream's documented workflow is human-in-the-loop, so its architecture is a useful foundation but not evidence that unattended final submission already exists. [S01, S02]

The scanner contains provider-specific discovery, URL and job-identity normalization, liveness checks, and deduplication. Reuse the design of a connector registry and retained source evidence. Validate each reused connector against current fixtures and access terms; a provider name in a registry is not proof of reliable current coverage. [S03]

The OpenRouter runner discovers free-priced models and supports model selection, but a manually pinned model path needs an additional independent free-price check. Its timeout and blacklist behavior should be replaced by measured timeouts, expiring health penalties, bounded retries, and account-level quota accounting. Its apply command prepares answers rather than carrying out a verified browser submission. The preparation script similarly produces application material, not a complete external commit. [S04, S05]

The artifact and answer-snapshot patterns are valuable: preserve the exact CV, JD, files, and selected answers used for each application. Keep this reproducibility, but use the new application's database as the operational state owner. Do not let upstream Markdown state and a second database independently decide whether an application was submitted. [S06, S07]

Reuse matrix

Upstream idea	Decision	New implementation requirement
Provider registry and public listing reads	Adapt	Provenance, policy metadata, health score, current fixtures
Canonical job identities	Extend	Requisition identity, employer identity, cross-source links
Career assessment prompts	Selectively adapt	Fact IDs, structured schemas, calibrated scoring
Application packet snapshots	Adopt concept	Immutable content hashes and manifest versioning
Free-model runner	Replace runtime behavior	Free-only gateway, privacy filters, quota governor
Interactive apply guidance	Use as research	Implement adapter contracts and receipt verification
Browser-specific quirks	Treat as hypotheses	Reproduce in fixtures before adopting selectors
Email aliases for repeated employer flows	Do not use as evasion	Canonical applicant identity and explicit duplicate policy

Fork versus clean implementation

Recommended: create a clean TypeScript monorepo and port small, useful, tested components with attribution. Maintain an upstream bridge package only if it reduces maintenance. Do not import a large script with hidden network side effects into a commit-capable service. A fork is also acceptable if P00 documents which state machine is authoritative and removes conflicting entry points.

Attribution must accompany copied code. Preserve the relevant MIT license and copyright notices; verify the pinned repository's license before each material import. Do not imply affiliation with Career Ops or copy branding as the new project's identity. A research-and-attribution document should identify file, commit, reason for reuse, local changes, and test coverage. [S01]

Where this project adds original engineering

The core contributions are a candidate evidence ledger; separate eligibility and fit assessment; cost-constrained model routing; reproducible document packets; a typed form adapter SDK; standing-authorization submission; ambiguous-commit reconciliation; a durable human-exception queue; and local-to-server parity tests. These contributions matter more than the number of agent personas. The GitHub demo should show a browser crash after a submission, recovery from the receipt, and no second application.

04. Candidate truth, Netherlands eligibility, and targeting

One factual profile, many tailored views

Maintain one canonical evidence-backed profile. Tailoring selects and orders facts; it does not create three inconsistent identities. The initial presentation is GenAI-heavy, with role lenses for applied AI/LLM engineering, AI-oriented software engineering, and selected ML/data roles where the work actually overlaps. A separate base CV is optional, not a requirement. Prefer a shared content model with versioned rendering profiles.

Store every material assertion with a stable fact ID, value, source document and location, verification status, effective dates, sensitivity class, and allowed wording. Preserve employer names, role dates, degrees, technologies, metrics, and delivery status. A source CV can contain errors, so extraction is not the same as verification. Conflicting dates, overlapping experience, impossible product timelines, and unsupported outcomes go into a fact-review queue.

Candidate-specific initial policy

This private operating profile reflects the owner's stated preferences and must be validated against the imported documents during onboarding. It is configuration, not sample data to publish on GitHub.

Field	Initial interpretation
Geography	Anywhere in the Netherlands; check the actual work location, not just company headquarters
Current employment	Markscape, remote employee, full-time with hourly-based pay
Notice	30 days; calculate earliest start relative to application date
Role scope	Applied AI, GenAI, LLM/RAG, relevant software and ML roles; adjacent roles when evidence supports them
Seniority	Assess duties, not just title; early-career and selected mid-level roles, not blanket senior applications
Languages	Use verified English and Dutch levels; English-compatible roles are priority
Compensation	Flexible, but immigration feasibility and the owner's approved salary answer remain separate
Permit	Orientation-year end date reported as 14 November 2026; verify and store effective-date rules
Sponsorship	Future sponsorship may be needed; employer openness and possible EOR arrangements are evidence fields

Experience wording and document emphasis

Separate hands-on LLM experience, paid professional AI experience, total employment, and full-time-equivalent experience. Compute non-overlapping months from approved timelines. Academic work and side projects may support hands-on experience, but cannot be silently recast as three years of full-time production employment. If the evidence supports it, wording such as approximately three years of hands-on LLM/RAG work can coexist with a separately accurate professional timeline. Do not hardcode that sentence before the fact audit passes.

Use relevant work examples first. The initial evidence map should prioritize Markscape's support-oriented agentic retrieval work and POEM's education-related AI work. Preserve distinctions between deployed capabilities and work

still in development, including due-diligence workflows. Projects such as DocIntel can fill a genuine evidence gap; they should not crowd out stronger professional examples. Specific achievements require their own approved metrics, not numbers inferred from the job description.

Immigration-aware rules, not immigration decisions

The IND's published 2026 reduced highly skilled migrant salary criterion is EUR 3,122 gross per month, excluding holiday allowance. Eligibility depends on the applicable route and circumstances; a flexible salary preference does not remove those requirements. Load effective-dated rules from authoritative sources, retain the checked date, and require owner or qualified-adviser confirmation for personal immigration conclusions. The system must refresh rather than carry a 2026 amount indefinitely. [S18]

Use separate facts for current right to work, permit expiry, future sponsorship requirement, and eligibility for a reduced criterion. Answer the exact question. Being currently authorized does not imply no future sponsorship, and needing future sponsorship does not necessarily mean no current right to work. If a binary question collapses these distinctions ambiguously, retain it for review instead of guessing.

Match employer entities against the IND recognized-sponsor register using legal names and identifiers where available. A recognized sponsor is not a promise to sponsor this vacancy. Absence from the register is not conclusive proof that every employment route is impossible. EOR/payroll employment is a separate possible arrangement that requires confirmation by the employer and provider; do not assert that Bluestone or another provider will accept a case merely because the owner is open to it. [S19]

Hard gates versus soft signals

Hard gates include a clearly required language level the candidate lacks, an explicit no-sponsorship condition incompatible with the candidate's needs, a closed vacancy, an impossible work-location requirement, or a genuinely essential qualification absent from the profile. Unknown sponsorship, an undisclosed salary, a preferred rather than required language, and an approximate experience target are not automatic rejections.

If a disclosed salary range straddles a relevant threshold, do not reject it because the lower bound is too low; record conditional feasibility. If even the upper bound is incompatible with the verified route, flag or reject according to policy. Never substitute the immigration floor for the candidate's desired salary. A required numeric expectation uses a preapproved value and unit; otherwise it becomes a missing-answer exception.

Explainable fit scoring

Apply eligibility gates before scoring. Proposed initial fit score: responsibilities 35 points, skill evidence 25, seniority alignment 15, relevant delivery/domain experience 15, practical preferences 10. Every component includes evidence and uncertainty. Rank eligible jobs using this fit score plus separate freshness and sponsorship-openness priorities. Start auto-preparation at a configurable 70/100 and keep a borderline queue around 55-69; validate thresholds against a labeled set before live use.

Show hard gaps, transferable skills, and unknowns. Do not call a score a selection percentage. The claim that applicants must always be among the first 10-30 to get an interview is not a dependable rule and must not be encoded. Prioritize prompt discovery without inventing applicant counts or discarding an excellent older vacancy solely because of age.

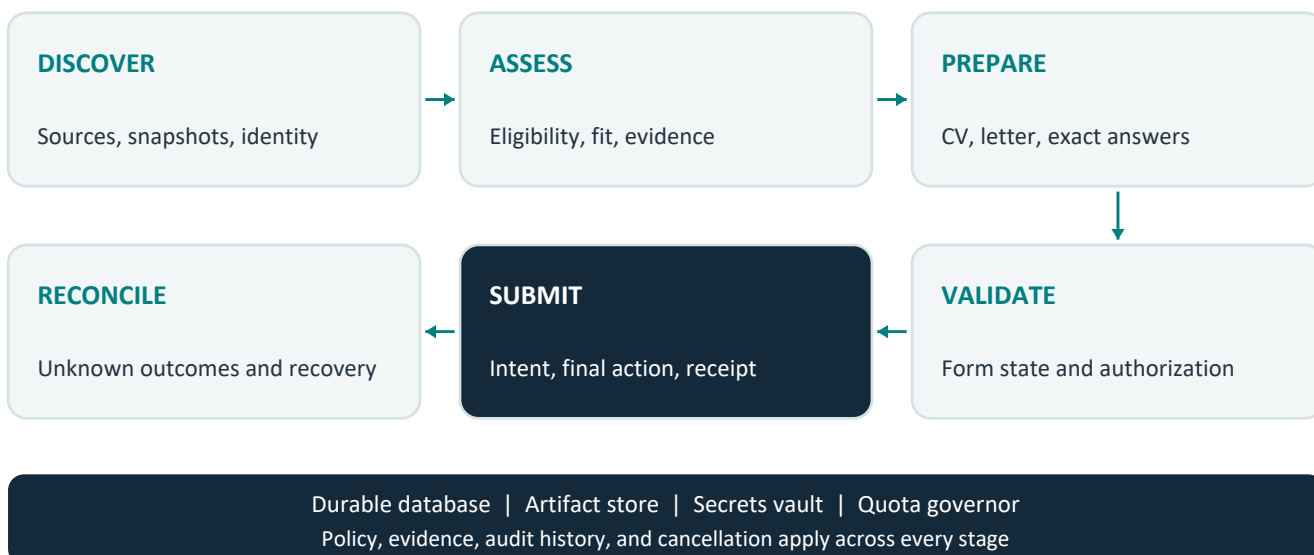
05. Reference architecture and repository boundaries

Architecture decision

Use TypeScript throughout the core, React for the dashboard, a small HTTP API service, a database-backed durable work queue, Playwright browser workers, and an OpenRouter gateway. Use structured schemas at every boundary. Keep Python optional for a narrowly isolated document utility only if the TypeScript rendering pipeline cannot meet a tested requirement. Pin supported runtime and dependency versions during implementation; do not select packages solely because this document names a category.

The operational database owns jobs, policy, tasks, attempts, and receipts. Agents produce typed proposals. The submission engine is the only component allowed to commit an application. An optional LangGraph graph can coordinate analysis and drafting, but it does not own external side effects. LangGraph resumption can re-enter node code; therefore a final browser click must never be hidden inside a replayable reasoning node. [S13]

Data path



Recommended monorepo

apps/	
web/	dashboard and onboarding
api/	authenticated application API
worker/	scheduler and durable task execution
browser-worker/	isolated browser and commit runtime
packages/	
contracts/	schemas, DTOs, error vocabulary
domain/	pure rules and state transitions
persistence/	SQLite and PostgreSQL repositories
discovery/	source connectors and normalizers
candidate/	facts, evidence, answer memory
ranking/	gates, score calculation, evaluation
model-gateway/	free-only routing and quota ledger
agents/	bounded analysis/drafting graphs
documents/	content AST, DOCX/PDF and QA
ats-sdk/	adapter contracts and fixtures
ats-adapters/	tested portal implementations
submission/	intents, commit gates, reconciliation
integrations/	optional email and notifications
observability/	redaction, events, measurements
tests/	
fixtures/	mock-ats/ fault-injection/ e2e/ goldens/
deploy/	
windows/	compose/ hybrid/ backup/
docs/	
adr/	phases/ runbooks/ evaluation/ security/ sources/

Local versus server persistence

Local mode uses SQLite on a non-synced local disk, WAL mode, short write transactions, foreign keys, checked durability settings, and a single scheduling authority. WAL still permits only one writer at a time and relies on same-host shared state. Do not place a live database in OneDrive or on a network share. Use a supported patched SQLite build and check upstream advisories at implementation time. [S20]

Server mode uses PostgreSQL with transactional leasing and explicit owner scoping. Queue selection can use row locks with SKIP LOCKED, but correct state transitions and fencing still need application-level design. Implement database-specific migrations and test both backends against the same repository contract. Do not pretend the SQL dialects are interchangeable. [S21]

Artifacts use a local content-addressed store in local mode and an S3-compatible store in server mode. Keep the same storage interface, checksums, retention policy, and encryption requirements. An export to the owner's existing CV and cover-letter folders is a copy of a completed artifact, not the authoritative mutable working location.

Keep the first runtime small

No separate message broker is necessary initially. Database queue transactions and an outbox are sufficient for the expected single-candidate workload. Avoid having both a polling scheduler and an agent supervisor claim the same task. If a future migration to Temporal becomes justified, write an ADR, preserve application identity and commit semantics, and prove replay compatibility. Do not make this migration a hidden prerequisite for submission.

Service ownership and permissions

The discovery process can fetch approved public sources but cannot read browser credentials. The model gateway can send minimized candidate facts but cannot submit forms. Document generation can read approved facts and write artifacts but cannot create accounts. The browser worker receives a job-specific execution packet, only the required secrets, and a narrow domain policy. The API authenticates the owner and persists instructions; it does not trust user interface state as authority.

06. Durable data model, queue, and state transitions

Core records

Record	Required fields and invariants
Candidate / profile_version	Owner ID, revision, effective time, source hashes; immutable published versions
Fact / evidence	Fact ID, typed value, evidence locator, review status, dates, permitted wording
Company	Canonical identity, names, domains, legal entity links; aliases retain provenance
Source / source_run	Connector, policy status, cursor, last success, errors, freshness metadata
Job / job_snapshot	Source job IDs, canonical application URL, normalized role/location, raw-content hash
Job_link	Cross-posting relationship, confidence, adjudication; never lose original links
Assessment	Gates, score components, gaps, source spans, evaluator and profile versions
Artifact / packet	Type, checksum, storage URI, provenance, QA outcome, generation settings
Answer / form_snapshot	Semantic key, exact question and option values, fact references, form fingerprint
Authorization	Owner, scope, effective/expiry times, daily cap, permissions, version and revocation
Application / intent	Stable job-candidate identity, packet hashes, policy version, state, uniqueness key
Attempt / receipt	Fencing token, start/end, ambiguity state, evidence locator, confidence and source
Task / outbox_event	Type, run-after, lease, attempts, state, deduplication key, transactional emission
Exception / outcome	Reason, required action, saved checkpoint, links, resolution and history
Model_request / audit_event	Redacted trace, budget reservation, usage, revisions, correlation IDs

Identity and deduplication

Primary application identity is candidate plus employer plus actual requisition identity. Prefer an ATS requisition ID; retain source IDs and normalized URLs. URL cleanup removes known tracking parameters only, not arbitrary query parameters that may distinguish jobs. Cross-postings are linked to one application target after evidence-based resolution. Same title does not prove same vacancy; same company can have multiple legitimate requisitions.

Use a strong unique database constraint for resolved application identity. Fuzzy company/title/location matches create a suspected-duplicate record, not an irreversible merge. Keep a merge/split audit trail. Reposted jobs need evidence of a new requisition before being treated as new. Do not use alternate email addresses to defeat employer duplicate controls.

Historical imports may be user-asserted submitted, receipt-verified, draft-only, or unknown. A cover-letter filename alone does not prove submission. The owner's explicit statement that earlier records were submitted can mark them user-asserted with that provenance. They still block accidental reapplication under policy, but they are excluded from today's newly confirmed count.

Application states

DISCOVERED -> NORMALIZED -> ASSESSED -> ELIGIBLE
ELIGIBLE -> PREPARING -> PREPARED -> INSPECTING -> READY
READY -> INTENT_RECORDED -> IN_FLIGHT
IN_FLIGHT -> CONFIRMED | DEFINITIVE_FAILURE | UNKNOWN
UNKNOWN -> RECONCILING -> CONFIRMED | NEEDS_REVIEW

Before commit: SKIPPED, CLOSED, DUPLICATE, NEEDS_INPUT,
CHALLENGE_REQUIRED, UNSUPPORTED, PAUSED, RETRY_WAIT.

After confirmation: RECEIVED -> INTERVIEW / REJECTED /
WITHDRAWN / OFFER / UNKNOWN_OUTCOME, tracked separately.

State transitions are pure domain functions with explicit preconditions. The workflow stores why a transition happened and which version of the rules made it. A browser timeout is not automatically a definitive failure. A withdrawal or deletion action is not part of retry handling. An outcome email does not overwrite the original application receipt.

Queue and leases

Claim ready work transactionally. Each claim receives a monotonically increasing fencing token and a lease expiry. A worker renews its lease while active and checks both the token and cancellation state before each privileged step. Transactions remain short; no database transaction stays open during browser navigation or model inference.

Pure preparation tasks may be requeued after a stale lease. Submission tasks with a recorded in-flight attempt must instead enter reconciliation. A second worker must not inherit a browser and click again because a lease expired. The commit executor must refuse concurrent execution for the same application identity; losing its valid lease closes or disables its committing context where possible. A network request already sent cannot be undone by fencing, which is why uncertainty must be preserved.

Use bounded retries by error class, exponential backoff with jitter, source-specific cooldowns, and dead-letter exceptions. Retry budgets survive restarts. A daily scheduler enqueues deduplicated work; it must not reset every failed task at midnight. Store timestamps in UTC and display daily counters using Europe/Amsterdam, including daylight-saving tests. Date-only permit and graduation fields remain date-only values.

Transactional events and integrity

Commit state change and outbox event in the same database transaction. Event delivery is at-least-once, so consumers must deduplicate by event ID. Store schema versions and migrate persisted task payloads deliberately. Audit events include actor, object, action, previous/current revision, and correlation ID. Hash chains can make accidental modification easier to detect, but do not claim they make a locally owned database tamper-proof.

07. Job discovery and fit pipeline

Source strategy

Start with company career pages and public ATS listing interfaces, followed by permitted feeds and owner-supplied URLs. Maintain a curated Netherlands employer watchlist, including international companies and English-speaking teams, instead of repeatedly asking a model to search the entire internet. Search is a discovery input; it is not a substitute for opening the actual current vacancy.

Greenhouse, Lever, and Ashby expose documented public listing paths. Public job reads do not imply candidate authority to use employer-facing submission APIs. Greenhouse and Lever submission integrations require appropriate employer credentials; a candidate normally does not have those. The default application path is therefore the employer's ordinary browser form, with authorized APIs supported only where credentials and permission genuinely exist. [S08, S09, S10]

For LinkedIn and Indeed, use owner-added links, permitted feeds, or an authorized integration where available. Resolve externally hosted applications to the real employer portal. Do not make restricted logged-in scraping a hidden dependency. Provide a bookmark/import workflow so a useful vacancy can enter the pipeline without the app controlling the job board itself.

Connector contract

Each connector implements source discovery or a known board crawl, pagination/cursors, normalization, liveness checks, and health reporting. Return structured records with original URL, canonical target, source job ID, employer, role, location, posted date if provided, first-seen date, observed date, and raw snapshot reference. Retain the original text before model extraction.

Every source has an allowed-access record, maximum request rate, concurrency, caching policy, recent response codes, and last successful parse. Honor server feedback. A 403 is not permission to change identity and retry; it disables or pauses the route pending review. A changed DOM should trip a parser-health alert before producing hundreds of empty descriptions.

Search taxonomy

Create role families with aliases: AI engineer, applied AI engineer, LLM engineer, GenAI engineer, AI software engineer, machine learning engineer with relevant deployment duties, NLP engineer, and selected AI-focused data scientist or solutions roles. Include adjacent titles only when responsibilities align. Search Dutch-language job titles too, while independently checking the working-language requirement.

Store positive and negative examples for each family. Exclude irrelevant uses of AI, roles dominated by a discipline unsupported by the candidate, and stale talent-pool listings unless the owner explicitly includes them. A talent pool is labeled as such and does not masquerade as a current vacancy. Deduplicate recruiter reposts against the company position when a reliable match exists.

Freshness and polling

Record `posted_at`, `updated_at`, `first_seen_at`, `last_seen_at`, and `closed_at` separately. Some boards refresh `updated_at` without reopening a job. Prefer feed delta cursors and conditional requests; otherwise poll high-value sources on a configurable 30-120 minute interval and lower-yield sources less frequently. Respect each source's allowed rate and do not create fake precision when posted dates are missing.

Build an initial backlog once, then process deltas. Revalidate liveness before document preparation and again before final form commit. If a listing disappears temporarily, use a short confirmation retry rather than immediately declaring closure. A form that says the vacancy is closed overrides a stale search result.

Ranking flow

First apply deterministic location, status, duplicate, language, and clearly incompatible conditions. Then extract responsibilities and requirements into a schema with supporting text spans. Compare these requirements against verified facts. Use a model for ambiguous semantic alignment, not for calculating experience dates or deciding what a numeric salary means.

The final score is computed in code from structured components and checks, not scraped from free-form prose. Store the raw structured assessment and reason codes. Unsupported model assertions such as assumed sponsorship or invented technologies cause validation failure. Re-evaluate when the JD or profile materially changes; a formatting-only source change should not trigger paid-like inference waste.

Coverage expansion based on evidence

Report how many suitable vacancies each source contributes, its duplicate rate, update lag, and unsupported application share. Build the next adapter for the highest-value repeated opportunity, not for the longest list of famous platforms. Company portals hosted on a common ATS can share an adapter while retaining employer-specific field variations. Difficult portals such as Workday or SAP SuccessFactors are later adapters with explicit support matrices, not claims of generic coverage.

08. Free-model gateway and bounded agent orchestration

Free-only is a runtime invariant

All model traffic goes through one gateway. It obtains the current model catalogue, checks the requested route's advertised pricing and capabilities, applies provider privacy requirements, reserves request budget, and only then sends inference. A model name containing free is not the only validation. A manually pinned model is subject to exactly the same checks as an automatically selected one.

Use decimal-safe price parsing. Inspect every applicable price dimension, including prompt, completion, request, and enabled tool or modality charges. Disable optional paid search tools and plugins. Restrict fallback routes to verified eligible free routes; fail closed when price or route information is unknown. Record the returned model/provider and reported usage, and trip a billing circuit breaker on unexpected nonzero cost. This reduces billing risk but cannot turn a third-party service into a contractual free-capacity guarantee. [S11, S12]

Quotas and the 50-application target

OpenRouter documents low limits for free usage, differentiated by account credit-purchase history, and warns that free-model capacity is unsuitable as a production availability guarantee. Limits can change. Read current documentation/account information at setup, maintain a local request ledger, and configure the actual daily and minute ceilings. Do not assume creating more API keys or accounts legitimately increases available capacity. [S11]

For planning, simulate both a restrictive 50-request daily budget and a more generous 1,000-request budget; these are capacity scenarios, not promised entitlements. Querying key credit status is not a reliable count of remaining free requests. Reserve budget before sending, reconcile usage afterward, and treat ambiguous network failures conservatively until the provider's accounting is known.

Example of a tightly constrained 50-request plan, assuming already-qualified opportunities and successful batches of four:

Inference work	Daily request allocation	Qualification
Semantic triage	5	Only uncertain jobs; deterministic filters do most work
Packet drafting	13	Up to 52 packets at four per successful batch
Independent packet critique	13	Same batch IDs, isolated item results
Novel form questions	4	Most routine fields use approved answer memory
Repairs	5	Bounded; unresolved cases are deferred
Health/quality probes	2	Do not waste the budget on constant inference checks
Unallocated reserve	8	Absorbs retries or additional questions

This arithmetic totals 50 requests, not 50 guaranteed submissions. Context limits, output truncation, provider errors, long letters, or many unfamiliar forms can make batches of four unsuitable. The scheduler must reduce batch size, reduce today's throughput, or wait for capacity rather than omit critique. Template-based truthful packets and cached approved answers can proceed without fresh inference when they still satisfy the quality gate.

Model selection and health

Use a small benchmarked pool: a fast classifier/extractor, a stronger factual writer, and a critic where a meaningfully independent route is available. Choose from current eligible models based on schema pass rate, claim accuracy, latency, context fit, and recent reliability. Do not hardcode an assertion that one named free model will remain available or best.

Use low-variance generation settings, explicit output schemas where supported, and a parser/validator even when schema output is advertised. Providers differ in structured-output support; require the necessary parameters or reject the route. Maintain expiring health scores, Retry-After handling when supplied, exponential cooldowns for 429/5xx errors, and a circuit breaker per route. A temporary timeout must not permanently blacklist a useful model. [S12, S14]

Privacy choices

Send only the facts needed for the current task. Contact details are usually inserted locally after drafting; passwords, cookies, API keys, passport scans, and unrelated email are never prompt content. Source job pages are untrusted data. Provider logging and retention differ, so expose provider-policy choices and record the accepted settings. A strict no-training or zero-data-retention choice may leave no free route available; then pause model-dependent work or use approved deterministic content, rather than silently relaxing privacy. [S12, S15]

Agent roles and tool permissions

Role	Output	Allowed capability
Requirement extractor	Typed JD requirements with spans	Read sanitized JD only
Match analyst	Evidence mappings and uncertainty	Read selected facts and requirements
CV/letter drafter	Content AST and claim references	Read evidence; propose text
Form-answer planner	Exact field proposals	Read form schema and approved answer memory
Critic	Specific defects and accepted claim IDs	Compare packet to facts/JD
Repair planner	Bounded revised proposal	Read defect list; no new facts
Workflow scheduler	Task transitions	Deterministic code, not an LLM persona
Commit executor	Submission attempt and evidence	Dedicated code with job-scoped authorization

Use a bounded graph: extract -> assess -> draft -> critique -> at most two repairs -> accepted or needs review. No recursive hiring of agents, open-ended debate, or autonomous tool installation. The critic cannot authorize a new fact merely because the drafter cited an ID. Deterministic numeric/date checks and claim-level support checks remain mandatory. Agent disagreement produces an exception or conservative wording, not a majority vote on truth.

Caching and reproducibility

Cache by sanitized JD hash, profile version, task type, schema version, prompt version, model identity, and policy-relevant settings. Cache exact accepted outputs and validation reports. Never reuse another job's company references or cover letter merely because the title matches. Batched output must include unique job IDs and pass a cross-item contamination test; a missing or duplicated ID invalidates the affected item only.

Store redacted execution traces, token usage, latency, error class, and validator results. Graph checkpoints are calculation state, not the authoritative application state. Version graphs and prompts; route paused executions to compatible versions or explicitly migrate them. Framework checkpointing alone does not guarantee compatible behavior after a deployment. [S13]

09. Document factory and application packets

Content before layout

Represent a CV as typed content: identity, summary, employment entries, education, skills, and optional projects. Each bullet includes its fact references and presentation constraints. Represent a motivation letter as salutation, role-specific opening, two or three evidence-linked contributions, motivation grounded in the actual role, practical availability where appropriate, and closing. Do not let a model emit arbitrary HTML, template code, file paths, or document macros.

The CV should retain a stable professional identity across applications. Tailor the summary, skill ordering, and most relevant bullets; do not rewrite job titles into the advertised title. Keep employment dates, degrees, employers, language levels, and factual outcomes fixed. One or two readable pages is an initial preference, not permission to delete essential context. Configure layout constraints explicitly and compare the tailored packet with the source profile.

Evidence and language rules

- Prefer work experience over personal projects when both support the requirement. Include a project only when it adds evidence the professional history lacks.
- Distinguish built, prototyped, researched, deployed, maintained, and currently developing. Do not promote an in-development workflow into a production achievement.
- Quantified outcomes require approved facts with units and a source. The job description is never a source for candidate accomplishments.
- Use the company's exact role and name from the verified vacancy. Avoid invented product knowledge, generic praise, unsupported cultural claims, or statements that the candidate has always wanted to work there.
- Keep visa explanations accurate and concise when requested. Do not lead every letter with visa anxiety or volunteer unrelated personal information.
- Use an approved approximately-three-years hands-on phrase only if the timeline audit supports it. Never inflate professional seniority to match a requirement.

Rendering and QA

Generate DOCX from a controlled template and PDF from the same content model. A practical initial pipeline is a tested DOCX library plus a pinned headless office converter, or a controlled HTML/CSS-to-PDF renderer alongside DOCX. Select one at P06 based on fidelity tests and deployment support. If the two outputs use different layout engines, test content equivalence explicitly rather than assuming it.

QA extracts text to verify company, role, required sections, dates, numbers, and absence of unresolved placeholders. Check page count, minimum font size, clipping, unexpected blank pages, file size, valid format, and upload restrictions. Render every new template and representative long/short packets visually in CI; run automated bounds checks for every generated PDF. A deterministic template is preferable to shrinking text until it fits.

Keep typography ATS-friendly: ordinary text, clear headings, logical reading order, no critical information hidden in graphics, and working URLs. Do not claim a universal ATS pass score. Preserve the original document unchanged. For each application, save the exact uploaded version even if a newer CV is generated later.

Immutable packet manifest

```
packet_id, candidate_id, application_id
job_snapshot_hash, profile_version, authorization_version
assessment_id, answer_memory_version, prompt_versions
cv: storage_key, sha256, mime_type, bytes, qa_report
letter: optional equivalent fields
answers: semantic_key, exact_text, exact_option_value,
         evidence_ids, approval_status, question_hash
claims: text_range, fact_ids, validation_result
created_at, accepted_at, generator_identity
```

The final packet is immutable. Editing an answer or swapping an attachment creates a new version and invalidates the old readiness check. Compute hashes from actual file bytes, not filenames. Export readable copies to the owner's Cover Letters and Tailored Resumes folders only after QA, using company, role, date, and a short stable job ID. Avoid overwriting similarly named applications.

Answer memory

Answer memory maps semantic questions to verified facts or approved owner preferences, with scope and validity dates. Examples include notice period, relocation willingness, employment status, education, current work authorization, future sponsorship, salary units, and portfolio links. Match the exact semantics, not just a shared keyword. A question about permanent authorization is different from one about current authorization.

Store the employer's actual option values and selected labels. A model may propose a mapping; code verifies that the selected option exists and the meaning matches. Do not default every yes/no question to yes. Optional demographic responses use a preapproved preference such as decline-to-answer where available. Unknown legal declarations, conflicts of interest, criminal-history questions, or mandatory attestations become precise exceptions rather than inferred answers.

Cheap fallback without misrepresentation

If inference is unavailable, use a previously approved role-family summary, reorder supported skills, select approved bullets deterministically, and generate a concise template letter using verified company/role information. Label how the packet was produced internally. It must pass the same factual and formatting gates. If the application asks a substantive unfamiliar question that the approved material cannot answer, defer that application while others continue.

10. Browser adapter SDK, accounts, and exception handling

Browser operation model

Use an app-owned browser profile or an explicitly selected supported browser session. Do not assume an already open personal Edge profile can be safely shared by multiple processes. First-run setup can launch a visible Edge-compatible browser for sign-in; unattended mode then uses the owned session. Browser state is a credential, not an ordinary cache. Store it encrypted and exclude it from logs, Git, and cloud-sync exports. Playwright explicitly warns that authentication state can permit impersonation. [S16]

Prefer accessible labels, roles, stable attributes, and verified form relationships. Use visual inspection only as a bounded fallback. An unconstrained screenshot agent should not search for a plausible Submit button on arbitrary pages. Every adapter declares supported host patterns, form variants, field types, account behavior, challenge detection, commit action, and receipt evidence.

Adapter interface

```
detect(url, page) -> adapter_id, version, confidence
inspect(page) -> FormSnapshot
classifyBlocker(page) -> Challenge | Login | Unsupported | None
ensureAccount(context, approved_identity) -> AccountState
planFields(form, packet) -> FieldPlan
fill(page, plan, artifacts) -> FillResult
validate(page, expected_state) -> ValidationReport
prepareCommit(page, intent) -> CommitReadiness
commitOnce(page, fenced_intent) -> AttemptObservation
verifyReceipt(page, attempt) -> ReceiptEvidence | Unknown
reconcile(context, application) -> Evidence | Unresolved
```

The adapter proposes observations and actions; the central submission engine owns authorization, state transitions, locking, and final classification. Detecting a Greenhouse page does not grant access to an employer API key. A generic fallback may fill only recognized, low-risk fields, but it cannot auto-submit until it meets the same adapter-level validation contract.

Form snapshot and fill verification

Capture canonical URL, employer and requisition identity, step index, field labels, HTML types, required flags, available choices, validation rules, conditional dependencies, upload constraints, and a structural fingerprint. Remove secrets and challenge tokens before persisting a diagnostic snapshot. Keep enough sanitized structure to reproduce a failure in a fixture.

After filling, read values back from the actual controls. Reinspect after actions that reveal conditional fields or auto-populate from the CV. Some portals parse a resume and overwrite names or experience, so verify the final values rather than trusting the initial fill. Upload completion must be observed; selecting a file is not proof the server accepted it. On multistep forms, validate each page and the final review step.

The form can change while preparation occurs. Compare question semantics and option sets before commit; a changed form invalidates the prior mapping. Never carry a numeric answer into a new salary unit or a yes/no answer into a differently worded sponsorship question without revalidation.

Guard against implicit submission as well as an explicit button click. Pressing Enter, choosing an autocomplete item, or a scripted field change can sometimes submit a form. Supported adapters must identify these paths and prevent application-finalizing events or requests during preparation. Arm a narrowly bound, one-use commit capability only after the durable in-flight record exists. Test this guard in dry-run mode and against implicit-submit fixtures. If an

adapter cannot distinguish routine field traffic from final submission reliably, it is not eligible for unattended commitment.

Accounts and authentication

Routine account creation is allowed when the standing policy covers it, the employer domain is verified, the identity is approved, and the flow has no unapproved payment or material additional commitment. Generate unique credentials and store them in the operating-system vault or server secret store. Never send passwords to a language model or put them in screenshot artifacts.

Email verification can use a connected mailbox only for a pending account challenge tied to the expected employer, recipient, time window, and domain. Treat email content as untrusted. Validate redirects and require that the final destination belongs to the pending account workflow. OAuth consent, unfamiliar MFA, passkeys, or CAPTCHA may require the owner. A manual-only account can still be useful after one successful setup.

CAPTCHA and other handoffs

Detection creates a CHALLENGE_REQUIRED exception, saves non-sensitive form progress, releases unrelated worker capacity, and displays a resume action. Do not persist challenge responses for reuse, outsource solving, or attempt to defeat portal defenses. For local mode, open the exact owned browser session. For server mode, provide a time-limited, authenticated view of that one session over a private channel; never expose raw browser debugging ports publicly.

Some sessions cannot be safely resumed after expiry or restart. In that case recreate the form from the saved packet and revalidate, but do not retry a final submission if the previous attempt is ambiguous. The challenge queue has expiry, last checked time, and a clear owner action. A solved challenge changes only that application's state and never widens the standing policy.

Exception taxonomy

Reason	Saved information	What continues
CAPTCHA / MFA	Session handle, expiry, safe screenshot	Other employers and read-only jobs
Missing answer	Exact question, options, affected applications	Jobs with known answers
Unsupported form	Adapter detection, sanitized structure	Supported adapters
Unclear sponsorship	Vacancy evidence and precise question	Other eligible jobs
Account blocked	Employer domain, status, retry limits	Other accounts
Model unavailable	Request type and budget state	Cached/deterministic work and discovery
Submission unknown	Intent, attempt, last observations	Reconciliation and unrelated jobs
Portal automation restricted	Access evidence and source status	Other permitted sources

Adapter rollout order

Use a mock ATS first, then one high-yield accessible portal family. Recrutee is a sensible first browser-adapter candidate given the owner's existing workflow, but the first choice must be validated against currently relevant jobs. Add Greenhouse, Lever, and Ashby browser variants next based on actual queue coverage. Treat Workday, SmartRecruiters, Teamtailor, custom company portals, and other families as separately tested increments, not a single generic milestone.

11. Final submission protocol and reconciliation

Standing authorization is the gate, not repeated clicking by the owner

The app offers review-only, fill-only, and auto-submit modes. The desired production mode is auto-submit. Onboarding records the owner's identity, permitted countries and role families, excluded employers, salary-answer policy, sponsorship wording, allowed account creation, approved disclosures, daily limit, domain restrictions, and expiry/revocation behavior. Store a signed or authenticated versioned policy record and an audit trail.

Before the final action, code checks: active owner authorization; current profile and policy versions; eligibility; vacancy liveness; resolved application identity; absence of historical or active duplicate; valid packet hashes; all required answers supported; current form semantics matched; completed uploads; no unresolved challenge; available submission budget; and exclusive committing ownership. Unknown is not equivalent to pass.

Intent snapshot

The immutable intent binds the application identity, destination origin and requisition, exact answer map, form fingerprint, document hashes, policy version, profile version, and adapter version. Hash a canonical serialization. An edit to any bound field requires a new intent. Store who or what authorized it and which deterministic checks passed.

The intent is a local transaction record, not proof of employer receipt. It supports reproducibility and safe reconciliation. Use a final fresh-state check immediately before commitment, without regenerating model text inside the committing interval.

Commit sequence

1. Claim application and obtain a valid fencing token.
2. Load immutable packet and verify all content hashes.
3. Open the correct form and complete/read back fields.
4. Recheck policy, identity, form, uploads, and challenges.
5. Persist INTENT_RECORDED with exact final-state snapshot.
6. Recheck cancellation, lease, and unchanged page state.
7. Persist IN_FLIGHT attempt durably before the final action.
8. Execute the adapter's single final submission action.
9. Observe acknowledgment, validation error, or uncertainty.
10. Persist evidence and CONFIRMED, DEFINITIVE_FAILURE, or UNKNOWN; publish the result through the outbox.

If step 7 fails, do not click. If the process dies after step 7 but before step 8, the record may be uncertain even if nothing was sent. This conservative ambiguity is intentional: local storage cannot atomically transact with an unrelated employer website. If the process dies after step 8, a restart must reconcile rather than repeat the click.

Evidence hierarchy

Strong evidence is an employer receipt identifying the role/application, an authenticated applicant-account record, or a job-specific success view with a retained correlation context. A confirmation email is useful when sender, recipient, employer, role or application ID, and time window align. A generic email saying account created is not an application receipt. An HTTP 200, disappeared button, spinner completion, or route navigation alone is insufficient.

Each receipt stores evidence type, normalized employer and job identity, observed timestamp, any application identifier, minimal text excerpt, optional redacted screenshot, content hash, and confidence rationale. Where a portal gives only generic thanks, require adapter-specific proof that the view belongs to this attempt and was not already present. Ambiguous evidence remains unknown.

Reconciliation without duplicates

Start with the existing browser response if available, then read-only account history, then correlated application email if connected. Retry observation with bounded delays. If a portal explicitly reports that the same requisition is already applied, preserve that evidence and determine whether it refers to the current attempt or an older application. Do not count an old application as a new success.

Only reattempt after definitive evidence that no application was accepted and a safe adapter-specific retry path exists. Lack of a receipt is not definitive evidence of failure. If the outcome cannot be established, leave NEEDS_REVIEW and do not sacrifice duplicate safety to meet a daily quota. Where an authorized submission API supports an idempotency key, use it; do not invent such support for browser forms.

Failure and cancellation cases

Event	Required response
Crash before intent	Preparation can resume normally
Crash after intent but before in-flight	Revalidate; no final action recorded yet
Crash after in-flight record	Reconcile even if click may not have occurred
Validation error clearly prevents acceptance	Record exact error; repair supported fields and create a new intent
Network timeout after final click	UNKNOWN, never automatic blind retry
Policy revoked before final click	Stop and invalidate readiness
Policy revoked after request sent	Record actual outcome; cancellation cannot unsend it
Two workers claim same job	Unique identity plus lease/fencing rejects second commit
Backup restored to older state	Reconcile the possible external-submission gap before enabling commits

The correct reliability claim

Call this an evidence-backed, effectively-once application workflow, not guaranteed exactly-once delivery to arbitrary external websites. The local engine can prevent many duplicate attempts, preserve ambiguity, and refuse unsafe replays. It cannot guarantee an employer's internal processing behavior. The engineering achievement is an honest uncertainty model with tested recovery, not an impossible distributed transaction claim.

12. Security, deployment, operations, and public release

Local installation

Ship a documented Windows installer or packaged launcher after the core is stable. The local dashboard binds to loopback, uses owner authentication or an unguessable local session credential, validates Origin/Host, and protects state-changing requests against cross-site attacks. Localhost alone is not authentication. Store secrets in a Windows-protected vault and operational data in a dedicated non-synced application-data directory.

The worker runs independently of the development agent. Provide start, pause, stop, status, diagnostics, backup, restore, and update commands. Configure an owner-approved scheduled startup or background service without opening unwanted windows. If the computer sleeps, powers off, or loses connectivity, local execution pauses; the dashboard must display that honestly. Do not promise 24/7 processing from a sleeping laptop.

Server profile

The production Compose profile contains API, web, scheduler/worker, isolated browser workers, PostgreSQL, artifact storage or an external S3-compatible endpoint, and a TLS entry point. Keep the browser version aligned with the Playwright package and follow supported container sandboxing guidance. Run with resource limits, a non-root posture where supported, read-only application files, and narrowly writable data locations. Do not expose database or browser-control ports to the public internet. [S17, S22]

A fully hosted profile can continue while the owner's PC is off, but it needs an awake server, storage, network, credentials, and occasional challenge handoff. Hosting, domain names, email services, and electricity may cost money even if inference is free. Provide resource measurement and cost configuration, not a promise of unlimited free hosting. Keep a single-owner deployment as the supported baseline.

Hybrid profile

Server-side discovery, preparation, and queue management communicate with a local browser worker through an outbound authenticated connection. Commands carry owner scope, application ID, intent hash, lease/fencing token, expiry, and a unique command ID. The worker rejects stale, replayed, wrong-owner, or wrong-origin commands. Use one authoritative database on the server; the local machine stores only a bounded encrypted execution cache and receipts awaiting acknowledgment.

The local worker must stop new commits if it cannot renew authorization/lease with the server. An already sent external request still enters reconciliation. Do not pretend an offline local queue and server queue can both independently commit the same application safely. Switching profiles requires an explicit drain and reconciliation step; move artifacts and profiles, but reauthenticate browser sessions when platform-bound secrets cannot be migrated safely.

Optional Gmail integration

Gmail is optional, not a dependency for the first successful submission. Begin with portal receipts and a manual receipt-import path. Later use the least sufficient read capability for confirmation and account verification. Gmail read scopes can be restricted scopes, and a label filter in application code does not turn a mailbox-wide OAuth grant into a label-only permission. Explain the actual scope to the owner. [S23]

Store refresh tokens in the vault, redact email bodies, and retain only necessary excerpts or identifiers. An externally configured OAuth app in testing can have short-lived refresh credentials, so test unattended renewal and document reconnect behavior. Public distribution or hosting of restricted-scope data can trigger verification or security-assessment requirements; evaluate the actual deployment instead of assuming a personal prototype's setup is a deployable public OAuth product. [S24, S25]

Threat controls tied to the workflow

Treat job pages, PDFs, model output, email, and repository content as untrusted. Strip active content, cap payload sizes, and prohibit instructions from these sources from changing tools, credentials, destination domains, or policy. Prevent server-side request forgery by rejecting local/private network targets and rechecking redirects and resolved addresses for imported URLs. Account-creation and verification links must remain inside a validated workflow.

Do not log full browser storage, Authorization headers, query-string tokens, or raw mailbox dumps. Encrypt sensitive artifacts at rest and in backup, with keys outside the backup payload. Use per-owner storage prefixes and authorization checks even in single-owner mode so a later extension does not depend on guessing object IDs. Separate a public synthetic demo from all real candidate data.

Retention and backup

Initial configurable defaults: short retention for debugging screenshots and raw HTML, longer retention for application packets and receipts, and indefinite retention only for explicit owner-managed records. A reasonable starting policy is 7 days for debug traces, 30 days for sanitized raw pages, and 12 months for application evidence, subject to owner choice and applicable obligations. Delete secrets immediately on revocation where feasible and document copies retained in encrypted backups.

Use database-consistent backups rather than copying a live SQLite main file while ignoring its WAL. Back up database, artifact manifest, artifacts, and configuration revisions together. Keep a separate recovery key process. Test restoration to a clean directory or instance. On restore, disable submission until the interval between backup and recovery has been reconciled against external receipts; old local state can otherwise create duplicates.

Observability and operator controls

The dashboard shows confirmed today, target gap, prepared queue, ambiguous submissions, needs-help items, next scheduled source run, browser worker health, quota remaining estimate, and the active authorization revision. Provide separate controls for pause discovery, pause preparation, pause new submissions, and emergency stop. A global stop revokes new execution leases and interrupts active contexts where safe; it cannot recall an already delivered application.

Structured logs and traces use application/task IDs, not personal data. Expose health/readiness checks and alert on stale workers, queue starvation, repeated adapter drift, unexpected cost, disk exhaustion, and growing unknown submissions. A daily summary explains shortfalls by source supply, eligibility, model budget, account friction, or unsupported portals. Never label a failed night successful because many CVs were produced.

GitHub portfolio standards

Publish architecture diagrams, ADRs, mock ATS fixtures, deterministic demo data, evaluation methodology, adapter support matrix, deployment guide, failure-recovery demonstration, screenshots, and a clear license. Include a security policy, contribution guide, issue templates, changelog, and attribution for upstream code. Use synthetic identities and receipt IDs in the public demo. An automated public demo may submit to the bundled mock ATS only.

The strongest demo is a five-minute sequence: discover a relevant synthetic job; explain fit; generate a supported packet; fill a dynamic form; submit; crash before local receipt persistence; recover without a duplicate; solve a queued manual challenge; show the audit trail. Publish actual benchmark counts and confidence intervals where meaningful. Do not present mock submission totals as real job-search results or claim interview improvements without enough outcome data.

Phase roadmap

Phase	Outcome	Depends on
P00	Repository audit, decisions, milestone ledger	This specification
P01	Runnable local skeleton and CI	P00
P02	Persistence, queue, and audit core	P01
P03	Candidate facts and standing policy	P02
P04	Discovery, identity, and historical import	P03
P05	Free-only gateway and evaluated ranking	P04
P06	Truthful document and answer packets	P05
P07	Mock ATS, browser SDK, and dry-run filling	P06
P08	Verified submission vertical slice	P07
P09	High-yield ATS adapter expansion	P08
P10	Accounts, challenges, and recovery UI	P09
P11	Optional email and receipt reconciliation	P10
P12	Bounded agents and model-cost optimization	P11
P13	Unattended local operation at target scale	P12
P14	Server and hybrid deployment parity	P13
P15	Security, fault, restore, and soak gates	P14
P16	Outcome learning and prioritization quality	P15
P17	Release packaging and portfolio publication	P16

Each phase below is a delivery contract. Tests and documentation are part of the phase, not tasks postponed to P15. P15 expands and independently validates the controls already built. The order intentionally reaches actual final submission at P08 before optional orchestration and broad deployment consume the project.

P00. Repository audit and implementation charter

Outcome: a reviewed starting point, one architecture, a concrete backlog, and a resumable execution ledger. No application submission is needed in this phase.

Inputs: this specification; any existing repository; the pinned Career Ops reference; the owner's source-document locations and preferences. If no repository exists, create the project in the owner-selected workspace, not inside personal document folders.

Work packages

- **P00-01: Inspect before changing.** Read applicable repository instructions, inventory files, detect package managers and runtimes, inspect Git status, and identify existing user changes. Summarize what can be reused. Do not discard existing work or assume the repository is empty.
- **P00-02: Record architecture decisions.** Write ADRs for database-backed workflow, SQLite/PostgreSQL parity, browser-owned external commits, free-only inference, canonical candidate facts, and three deployment profiles. Explain why Temporal and unrestricted swarms are deferred.
- **P00-03: Establish the requirement ledger.** Create records for R01-R12, every phase, and each acceptance gate. Each record has status, dependencies, evidence paths, last tested commit, and unresolved external prerequisites.
- **P00-04: Define scope and support claims.** Create an initial adapter matrix with mock-only, planned, fixture-tested, dry-run-tested, and live-verified levels. Separate discovery support from submission support.
- **P00-05: Plan reference reuse.** Record upstream commit and license, candidate files, modifications needed, and tests required before import. Do not run imported scripts with real credentials during the audit.

Artifacts and implementation boundaries

Deliver README, architecture overview, ADRs, a phase ledger, requirement traceability table, private-data exclusions, and a risk register limited to concrete design concerns. Add example configuration with synthetic values. The owner's CV, phone, email, permit evidence, browser state, historical applications, and API key must not enter committed fixtures.

Choose and pin supported versions during implementation after checking their official compatibility information. Record the installed browser, runtime, database engine, and document converter versions in a machine-readable diagnostics record. Do not state the project is production-ready at this point.

Acceptance gates

1. Every R01-R12 requirement maps to at least one implementation phase and a verification method.
2. The architecture contains exactly one authoritative application state store per deployment.
3. The first genuine submission milestone is P08, not hidden behind a later optional agent framework.
4. Existing changes are preserved and the audit explains any conflicts requiring owner input.
5. A clean-room reader can identify how to start, where private configuration belongs, and how progress is recorded.

Evidence and handoff

Save the audit, dependency matrix, ADR index, and phase ledger under docs. Mark P00 complete only when these are present and internally consistent. The handoff identifies the next smallest P01 ticket, selected stack, unverified assumptions, and exact files to inspect. Estimated durations may be recorded as rough planning ranges, but never become reasons to skip a gate.

P01. Runnable skeleton, developer experience, and CI

Outcome: a clean clone starts a local dashboard, API, and worker in synthetic mode, with passing checks and a reproducible development environment.

Depends on: P00. **External prerequisites:** none for synthetic mode; do not require an OpenRouter key, Gmail login, or real employer account just to run tests.

Work packages

- **P01-01: Scaffold boundaries.** Create the apps and packages listed in chapter 05, but implement only the necessary initial modules. Empty directories are not features. Establish shared schemas, typed errors, structured logging, and configuration validation.
- **P01-02: Define runtime profiles.** Support explicit demo, local, and server configuration types. Default a public clone to demo with synthetic data and external submission disabled. Keep the user's private installation capable of explicit auto-submit configuration later.
- **P01-03: Build health surfaces.** Add version, liveness, readiness, and worker-heartbeat endpoints. Readiness must fail for unavailable required persistence, not merely return HTTP 200 because the process runs.
- **P01-04: Create the dashboard shell.** Show source jobs, application stages, exceptions, workers, and settings using synthetic data. Add accessible keyboard navigation and clear empty/loading/error states.
- **P01-05: Establish CI.** Run formatting, linting, type checking, unit tests, dependency checks, and a production build. Use deterministic fixtures and pin the package lockfile. No test may send a real application.

Command contract

Implement documented commands equivalent to `dev`, `build`, `lint`, `typecheck`, `test:unit`, `test:integration`, `test:e2e`, `doctor`, and `demo:reset`. A command that is not yet implemented should fail clearly, not return success with a placeholder message. Demo reset is restricted to a validated synthetic data directory and must never remove an arbitrary configured path.

The diagnostics command reports runtime versions, directory write access, browser availability, disk capacity, database connection, converter availability, and masked configuration status. It must not print secrets. Provide a Windows-native path through the setup instructions without requiring the owner to debug Linux assumptions.

Acceptance gates

1. A clean clone installs and starts using the documented process.
2. The API, worker, and UI show consistent version and environment information.
3. Invalid or missing configuration causes an actionable startup error.
4. Secret-like fixture values are synthetic and a repository scan finds no real candidate data.
5. CI passes without external paid services or candidate accounts.
6. The UI never labels sample data as real submitted applications.

Evidence and handoff

Save a clean-start transcript with secrets redacted, CI artifacts, dashboard screenshots, and command documentation. Add unit tests for configuration precedence and environment validation. Handoff to P02 lists the repository interfaces that persistence will implement, not a suggestion to build the whole app inside an API route.

P02. Persistence, durable work, and audit events

Outcome: application work survives restarts, concurrent workers cannot independently own the same commit, and every material transition is inspectable.

Depends on: P01. Implement SQLite first, then the PostgreSQL repository contract in this phase so later domain code does not accidentally depend on SQLite behavior.

Work packages

- **P02-01: Schema and migrations.** Create owner, candidate version, source, job, application, task, attempt, artifact, authorization, exception, and audit/outbox records. Include foreign keys, uniqueness constraints, timestamps, schema versions, and migration tests.
- **P02-02: Repository interfaces.** Implement transaction boundaries and typed query methods. Keep backend-specific lease SQL inside persistence, not scattered through workers.
- **P02-03: Queue worker.** Implement claim, heartbeat, lease renewal, retry-wait, cancellation, dead letter, and success acknowledgment. Add application-level and domain-level concurrency limits.
- **P02-04: Pure state machine.** Reject invalid transitions and stale revisions. Distinguish retryable preparation from externally ambiguous attempts.
- **P02-05: Outbox and audit.** Atomically persist state plus event, then deliver events idempotently. Redact sensitive fields by schema.

Implementation details

Use a stable task deduplication key derived from task type and logical input revision. A source poll can be repeatable, but two identical polls should not create duplicate preparation jobs. Store retry count and next-attempt time; do not lose them when the worker restarts. Use a fake clock in tests to avoid timing-dependent sleeps.

For SQLite, validate WAL activation, busy timeout, foreign keys, and durable commit behavior. For PostgreSQL, prove transactional claiming with multiple concurrent clients. Use a single scheduler authority locally, and database-mediated ownership on a server. Implement a migration lock so two starting processes cannot both apply schema changes.

Acceptance gates

1. Concurrent claims produce one owner for each task and one committing owner per application.
2. Killing a worker during a preparation task results in recoverable requeue after lease expiry.
3. Killing a worker after an in-flight attempt creates reconciliation work, not a fresh submission task.
4. At-least-once outbox delivery produces no duplicate downstream side effect.
5. The same repository contract suite passes on SQLite and PostgreSQL.
6. A migration from the previous schema retains application identity and audit history.
7. Disk-full and database-unavailable errors prevent new external commits.

Evidence and handoff

Deliver schema diagrams, migration scripts, state-transition tests, concurrency results, and a queue-inspection screen. Include a test that starts two workers against the same application and proves only one receives a valid commit capability. P03 can now safely store facts and policy without introducing a second persistence layer.

P03. Candidate evidence and standing authorization

Outcome: one versioned candidate profile and a precise policy that permits routine final submission while isolating unknowns.

Depends on: P02. Reading and importing the owner's supplied documents is in scope; changing their historical facts is not.

Work packages

- **P03-01: Source ingestion.** Import DOCX/PDF files without modifying originals. Extract text, tables, hyperlinks, and source locators; store file hashes and parsing quality. If extraction is uncertain, surface it instead of silently dropping sections.
- **P03-02: Fact ledger.** Create typed employment, education, skills, language, project, metric, availability, and work-authorization facts. Mark extracted, owner-asserted, verified, conflicting, or expired status explicitly.
- **P03-03: Chronology validation.** Compute experience in non-overlapping months, track full-time/part-time distinctions, and flag inconsistent dates or technologies assigned to implausible periods. Never repair these by inventing substitute tools.
- **P03-04: Owner policy.** Build onboarding for role scope, location, sponsorship wording, salary answers, account creation, optional disclosures, blocked employers, and daily submission limit. Provide review-only, fill-only, and auto-submit modes with clear consequences.
- **P03-05: Answer memory.** Store approved semantic answers with valid dates, scope, exact meaning, and evidence. Add a reusable review mechanism that can resolve the same question across several pending applications.

Practical onboarding behavior

The owner should review a compact set of material ambiguities once, not hundreds of repeated forms. Previously stated preferences can prefill configuration, but an unknown legal or factual answer remains unknown. Display the difference between current work permission and future sponsorship. Configure the reported permit expiry as a date, not a vague narrative note that a model must interpret each time.

Keep salary willingness separate from a required numeric expectation and a route-dependent feasibility threshold. The UI can offer an approved answer range and unit policy, but must not invent a salary on the owner's behalf. A changed policy affects future commitments; existing packets and attempts retain their original versions for audit.

Acceptance gates

1. Every material candidate claim is traceable to a source or explicit owner assertion.
2. Overlapping employment does not inflate computed experience.
3. A profile revision invalidates affected prepared packets while preserving historical ones.
4. Revoking auto-submit blocks new commits immediately at the policy gate.
5. Missing salary or ambiguous sponsorship answers create targeted exceptions without blocking unrelated applications.
6. The public repository contains synthetic examples only.
7. The user can export and inspect the exact active authorization in plain language.

Evidence and handoff

Save private import reports locally, not in Git. Commit only sanitized fixtures, validators, profile schemas, and tests. Provide a candidate-profile summary showing which facts can be used in generation and which are withheld pending review. P04 receives a stable candidate ID and policy revision, not raw unstructured CV text as its only source of truth.

P04. Discovery, job identity, and historical imports

Outcome: the system finds real relevant opportunities from supported sources, records freshness, and avoids rediscovering historical submissions as new work.

Depends on: P03. Public source reading can be tested independently of model availability.

Work packages

- **P04-01: Connector SDK.** Implement source configuration, pagination, incremental polling, rate policy, raw snapshots, normalized output, and health metrics. Add fixture replay mode and robust timeout/error classes.
- **P04-02: Initial connectors.** Implement a small set of high-yield public ATS listing connectors and owner-added URL ingestion. Verify current live read-only samples and retain source attribution.
- **P04-03: Canonical identity.** Resolve requisition IDs, application URLs, employers, source aliases, and likely cross-postings. Keep suspected duplicates reversible.
- **P04-04: Historical import.** Import existing application records and document-folder metadata. Preserve user-asserted submitted status separately from independent receipts. Never infer success solely from a letter file.
- **P04-05: Watchlist and freshness.** Add Netherlands company lists, role aliases, first-seen/posted/updated times, closure detection, and source-specific polling schedules.

Data quality controls

A source response with a dramatic unexplained drop in job count, empty descriptions, or changed fields triggers a parse-quality warning. Do not automatically mark all prior jobs closed after a single bad parse. Preserve the raw response or a sanitized snapshot for debugging. Normalize locations into country/city/remote policy while keeping the original location text.

Support manual merge/split resolution for ambiguous duplicates. A confirmed application on one cross-posting must suppress other links to the same requisition. Two genuine roles at one employer remain distinct. Search-result redirects and tracking URLs must resolve through the same URL safety checks as direct imports.

Acceptance gates

1. Repeated polling of the same feed creates no duplicate jobs or application work.
2. Pagination and delta cursors do not omit records across fixture boundary cases.
3. Closed, temporarily missing, and parser-failed states remain distinguishable.
4. Cross-postings map to one application where identity is established.
5. Historical user-asserted submissions block accidental repeat applications without inflating new confirmed totals.
6. Source 403/429 responses activate the configured pause/backoff behavior.
7. At least one real current vacancy can be read, normalized, and displayed with source evidence, without submitting it.

Evidence and handoff

Deliver connector fixture sets, normalization examples, deduplication tests, a source health dashboard, and a coverage report. Record the exact sources and dates checked, not a generic claim that all portals work. P05 receives normalized job snapshots, source spans, and explicit unknown fields for evaluation.

P05. Free-only inference and evaluated matching

Outcome: a zero-paid-fallback model gateway and a tested eligibility/ranking pipeline that distinguishes facts from uncertain interpretation.

Depends on: P04. A real OpenRouter key is optional for offline fixture tests but required for live inference validation.

Work packages

- **P05-01: Gateway policy.** Implement model discovery, current price/capability checks, route allowlists, privacy settings, account-level budget reservation, timeout handling, and a no-paid-tools default.
- **P05-02: Resilience.** Add schema validation, bounded repair, expiring model health, rate-limit backoff, daily-budget exhaustion states, and redacted request logs.
- **P05-03: Deterministic gates.** Implement location, language, duplicate, closure, salary-feasibility, and explicit sponsorship restrictions from typed facts and requirements.
- **P05-04: Semantic assessment.** Extract requirements with evidence spans and map them to candidate fact IDs. Compute the final weighted score in code.
- **P05-05: Evaluation dataset.** Create at least 50 labeled, sanitized examples covering strong matches, adjacent roles, hard disqualifiers, ambiguous sponsorship, seniority mismatch, and misleading titles. Keep a holdout subset untouched during prompt tuning.

Quality criteria

Initial target: all hard-disqualifier cases in the curated safety set are correctly routed; at least 90% precision among auto-eligible predictions on the initial labeled evaluation; and zero invented candidate facts. Report sample size, per-class errors, and uncertainty rather than claiming a universal accuracy rate from 50 examples. A failed metric requires correction or a stricter auto-submit eligibility boundary, not removal of difficult test cases.

Models must return structured gap and evidence fields. Unsupported claims, malformed IDs, missing source spans, or inconsistent numeric interpretations fail validation. Use a deterministic fallback for obvious cases. The free-router option can be explored, but production selection must retain sufficient route/capability evidence for the quality and cost policy.

Acceptance gates

1. A manually pinned paid model is rejected in free-only mode.
2. An unknown price field or paid tool configuration is rejected before inference.
3. Quota reservations remain consistent under concurrent requests and restarts.
4. A 429 does not trigger account/key rotation or unbounded retry.
5. A schema-valid but unsupported candidate claim is rejected.
6. The ranking evaluation report includes held-out errors and score-component explanations.
7. Lack of an eligible free route pauses inference but leaves discovery and deterministic work operational.

Evidence and handoff

Deliver gateway tests, model-policy configuration, evaluation data and report, redacted live probe results if credentials exist, and an account-budget dashboard. If live inference cannot be tested, mark that check externally pending and retain a fully working mocked gateway. Do not mark live free-model reliability proven from fixtures alone. P06 receives accepted fact mappings and a validated text-generation interface.

P06. Tailored CVs, letters, and answer packets

Outcome: application-ready documents that are truthful, readable, reproducible, and tied to a specific job and profile revision.

Depends on: P05. Use the source profile from P03, never an agent's memory of the CV.

Work packages

- **P06-01: Content schemas.** Define CV, letter, claim, and answer ASTs with fact references, length constraints, and template versions. Ensure templates contain no executable model-authored markup.
- **P06-02: Generation.** Implement professional-evidence-first bullet selection, role-specific summaries, concise motivation letters, and semantic answer reuse. Keep dates, employers, degrees, and metrics immutable without a new approved fact.
- **P06-03: Validation.** Check claim support, years-of-experience wording, company/role correctness, language requirements, tense/delivery status, and absence of placeholders or another employer's name.
- **P06-04: Rendering.** Produce DOCX and PDF using the chosen pinned pipeline. Add extraction checks, layout bounds, page limits, visual golden tests, and upload-size constraints.
- **P06-05: Packet store.** Write immutable manifests and content-addressed artifacts; export user-readable copies without overwriting originals.

Representative test packets

Include a short JD, a long JD, a role with no meaningful match, a CV with long employer names, a two-page CV, a letter with optional sponsorship text, a required character-limited answer, and a job whose company name resembles another employer. Test accented characters and names, long URLs, date formats, bullet wrapping, and converter failure.

Generate a change summary for the owner: reordered skills, selected evidence, summary changes, and excluded unsupported claims. The system should be able to explain why a work example was selected instead of a side project. Preserve the original CV and previous packets so an interview can be prepared against the exact submitted version.

Acceptance gates

1. Every material generated claim links to approved evidence and passes independent validation.
2. A prototype cannot be described as deployed in the test set.
3. Hands-on and professional experience totals remain distinct.
4. All representative PDFs render without clipping, unreadable text, or missing sections.
5. DOCX and PDF contain equivalent candidate facts and company/role references.
6. A changed artifact hash invalidates an existing readiness/intent record.
7. A model outage still permits approved deterministic packets, while unresolved substantive questions are deferred.

Evidence and handoff

Commit synthetic golden documents and QA reports, not the owner's real resume. Store private application packets in the configured artifact store. Provide a side-by-side review screen and a manifest viewer. P07 receives immutable artifacts, exact answer proposals, and validation reports suitable for browser use.

P07. Mock ATS and browser filling contract

Outcome: a controlled browser can inspect, fill, upload, and validate dynamic application forms without making real external submissions.

Depends on: P06. This phase creates the test environment that makes later submission engineering credible.

Work packages

- **P07-01: Mock ATS server.** Build synthetic job pages and forms with server-side application records, unique application IDs, configurable failure points, uploads, conditional questions, multi-step flow, and explicit receipt pages.
- **P07-02: Browser runtime.** Launch owned browser contexts with resource limits, isolation, domain policies, trace redaction, cancellation, and safe session storage. Add visible local mode for owner observation.
- **P07-03: SDK.** Implement the interfaces in chapter 10, including form snapshots, field plans, read-back validation, and receipt observations.
- **P07-04: Fill primitives.** Support text, email, phone, textarea, select, radio, checkbox, date, file upload, autocomplete, and character-limited answers. Unsupported controls must be reported explicitly.
- **P07-05: Challenge fixtures.** Model CAPTCHA/MFA states with synthetic challenge screens, without implementing bypass. Persist exceptions and demonstrate other jobs continuing.

Difficult fixtures

Include resume parsing that overwrites a field, a hidden conditional sponsorship question, a required field revealed after selection, an upload that fails after file selection, an autocomplete that requires selecting a real option, duplicate labels on different steps, a changed salary unit, a disabled submit button, and a misleading success-like banner on the initial page.

The mock server must know whether an application was actually received so tests can assert reality independently of the UI. It must distinguish account creation from application submission. Keep all fixture identities synthetic and prevent the test environment from navigating to real employer domains.

Acceptance gates

1. Filled values read back exactly, including underlying option values where relevant.
2. Upload validation distinguishes selected, uploading, accepted, and failed states.
3. Conditional fields are reinspected and required answers cannot be missed.
4. A changed question invalidates an old answer plan.
5. Challenge detection creates a resumable exception without stopping other queue items.
6. A generic banner or HTTP 200 cannot create confirmed-submission state.
7. The dry-run mode cannot accidentally invoke a final external commit, including implicit submission through Enter or a field-change event.

Evidence and handoff

Deliver mock server fixtures, SDK documentation, browser tests, redacted screenshots, and coverage by field type. Demonstrate a complete dry run from a discovered synthetic job to READY. P08 should add the central commit protocol, not replace these checks with a generic click command.

P08. Actual submission vertical slice

Outcome: the first complete, verified application submission path, including final click and recovery from uncertainty. This is the first central product milestone.

Depends on: P07. A genuine live test additionally requires an eligible real vacancy, a configured candidate profile, standing authorization, valid session where needed, and a supported permitted portal path. Do not submit a fake application to test production.

Work packages

- **P08-01: Policy gate.** Implement the complete precommit checklist and immutable intent binding. Verify profile, policy, destination, form, answer, document, duplicate, and budget constraints.
- **P08-02: Commit engine.** Persist intent and in-flight attempt before the final action. Enforce exclusive commit ownership, cancellation checks, and exact adapter routing.
- **P08-03: Receipt verification.** Implement strong success, definitive validation failure, and unknown-outcome classification. Store evidence with job/application correlation.
- **P08-04: Reconciliation.** Recover from an in-flight record without replaying the final click. Read the mock ATS account/receipt record and resolve accepted versus unknown states.
- **P08-05: First real adapter.** Choose one high-yield portal family from current candidate opportunities. Implement and test its actual final submission and receipt behavior, then run a genuine controlled application under the recorded policy.

Mandatory crash sequence

Test a worker kill before intent, after intent, after in-flight persistence but before click, immediately after the server accepts, after the success view loads but before receipt persistence, and after receipt persistence but before queue acknowledgment. Run the suite with process restarts and concurrent workers. The mock server's application count is the ground truth for duplicate detection.

No retry helper may automatically wrap the entire commit function. Browser locator retries for precommit inspection are different from replaying an application POST or final button action. Record exactly where automation becomes externally consequential.

Acceptance gates

1. Auto-submit mode completes the mock workflow without a per-application owner click.
2. Each confirmed state has independently correlated receipt evidence.
3. Every crash case avoids blind duplicate submission; ambiguous cases remain explicit.
4. Revocation immediately before commit prevents the final action.
5. A stale worker cannot acquire a new valid commit capability.
6. At least one genuine, appropriate live submission is receipt-verified before the adapter is labeled live-verified.
7. If a live prerequisite is unavailable, code completion and live verification are reported separately, and unrelated engineering continues.

Evidence and handoff

Deliver the complete synthetic recording, crash-test report, private live receipt when available, and support-matrix update. This milestone must not be described as complete merely because the form was filled. P09 expands a proven submission contract to more portals; it does not redefine submitted to mean prepared.

P09. High-yield portal coverage

Outcome: several independently tested portal adapters share one reliable SDK, with accurate coverage labels and graceful unsupported cases.

Depends on: P08. Expand based on the actual backlog of eligible Netherlands jobs, not on the number of platform logos desired for the README.

Work packages

- **P09-01: Coverage analysis.** Group eligible jobs by portal family, employer-specific variant, account requirement, and current failure reason. Prioritize adapters by expected additional suitable submissions per engineering effort.
- **P09-02: Adapter increments.** Add the next high-yield families, normally from Recrutee, Greenhouse, Lever, and Ashby browser forms. Treat each adapter as its own ticket with detection, field mapping, validation, commit, and receipt implementation.
- **P09-03: Variant fixtures.** Capture sanitized real form structures and rebuild representative variants in the mock environment. Include employer-added questions and localized labels.
- **P09-04: Drift detection.** Monitor structural fingerprints and missing expected controls. Pause only the affected adapter/version when its contract breaks.
- **P09-05: Support matrix.** Publish discovery, inspection, fill, account, commit, receipt, and reconciliation coverage separately, with last-tested dates and known limitations.

Adapter quality rules

Avoid monolithic selector code with a growing list of employer names. Share stable field primitives; keep platform-specific logic small and explicit. A supported platform can still contain an unsupported employer-specific question or custom iframe. Domain routing must be exact enough that a lookalike website cannot obtain candidate data.

Use no employer-facing API credentials unless the deployment actually has authorization. Direct candidate browser flow remains the default. Do not treat a public listing endpoint as a documented public application endpoint. A custom company's form receives its own allowed-origin and evidence configuration even if common primitives are reused.

Acceptance gates

1. Every added adapter passes the common SDK contract and portal-specific fixtures.
2. Each supported variant has at least one successful synthetic receipt and relevant negative cases.
3. A new unknown mandatory field stops that application before commitment.
4. A drifted adapter cannot degrade into unrestricted generic clicking.
5. Each live-verified label has a genuine private receipt and date; untested variants remain explicitly provisional.
6. At least two portal families complete the full synthetic pipeline without special workflow code outside the adapter boundary.

Evidence and handoff

Deliver the coverage report, fixtures, adapter documentation, and measured preparation/submission times by family. Expand toward more platforms only after these gates pass. Workday and other complex account-heavy portals can be backlog items with precise reasons, not fake checked boxes. P10 adds robust account and human-intervention workflows to increase usable coverage without weakening the submission contract.

P10. Accounts, secure handoffs, and exception resolution

Outcome: common account flows work under policy, genuine human-only steps are easy to resume, and one blocked application does not halt the system.

Depends on: P09. Optional mailbox automation is completed in P11; this phase supports manual verification and already-authenticated sessions.

Work packages

- **P10-01: Account registry.** Map candidate identity to employer domain and applicant account. Store only vault references in the database, with creation status, session validity, and last verification time.
- **P10-02: Signup workflow.** Fill supported ordinary signup forms using approved identity, generate unique credentials, and store them securely before a workflow can depend on them. Detect verification, MFA, payment, and unusual agreement steps.
- **P10-03: Challenge handoff.** Preserve progress, create a scoped session handle, expose a visible local resume route, and expire stale handles. Do not implement CAPTCHA bypass.
- **P10-04: Exception inbox.** Show the exact question/blocker, affected job, suggested supported answer if any, evidence, and actions: resolve, defer, skip, or open session.
- **P10-05: Propagation and resumption.** An approved answer can resolve semantically equivalent pending questions within its scope. Revalidate the entire affected form before commitment.

Session and authority details

Only one actor controls a handoff session at a time. Pause automation while the owner is interacting, then require a clear resume event before the worker continues. The resumed worker reacquires a lease and rechecks policy, field state, and expiry. A manual interaction that submitted the form must be detected and reconciled, not followed by another automated submission.

Avoid creating multiple employer accounts because an earlier signup response was lost. Account creation has its own pending/confirmed/unknown states, canonical email, and verification evidence. The same external-ambiguity principle applies, even though the application commit record is separate.

Acceptance gates

1. Supported signup stores credentials safely and never exposes them to model prompts or logs.
2. A lost signup response does not automatically create a second account.
3. CAPTCHA/MFA generates a usable handoff while unrelated queue work continues.
4. Handoff URLs/handles expire, are owner-bound, and cannot control another session.
5. User interaction and automation cannot race the final submit.
6. Resolving one answer does not incorrectly propagate to differently worded legal or sponsorship questions.
7. A resumed expired form is rebuilt safely, while an ambiguous prior commit is reconciled first.

Evidence and handoff

Deliver account-state tests, local challenge demonstrations, exception UX screenshots, and a redaction audit. Verify the dashboard's message is specific: for example, select whether future sponsorship is required, not an unhelpful generic failed label. P11 may automate narrow verification/receipt reading, but a disconnected mailbox must not break these manual routes.

P11. Email integration and receipt reconciliation

Outcome: optional email can resolve account verification and application outcomes without becoming an unrestricted mailbox agent.

Depends on: P10. Requires owner OAuth setup for live validation; all parsers and correlation logic remain testable with synthetic messages.

Work packages

- **P11-01: OAuth lifecycle.** Implement least-sufficient scopes, secure token storage, refresh, disconnect, revocation handling, and actionable reconnect states. Document testing-mode versus deployed-app limitations.
- **P11-02: Narrow message acquisition.** Query only relevant time windows and application/account contexts, fetch minimal fields, and avoid retaining unrelated email. Explain that these application filters do not narrow the underlying OAuth grant.
- **P11-03: Correlation.** Match messages to pending applications using expected recipient, employer/sender domain, role or requisition ID, application ID, and timing. Preserve uncertainty where several jobs could match.
- **P11-04: Verification links.** Follow only links bound to a pending signup and validated destination chain. Reject unexpected redirects or instructions to disclose secrets.
- **P11-05: Outcome updates.** Classify receipt, interview, rejection, or ambiguous message with supporting evidence. Do not automatically reply to recruiters or accept interview times in this scope.

Reconciliation behavior

An account-welcome message is not an application receipt. A recruiter newsletter is not an interview. A rejection proves some application existed, but may not establish that the current ambiguous attempt was received today. Maintain linkage confidence and provide an owner correction path. Never discard the original receipt because a later outcome changes.

If Gmail is unavailable, fall back to portal evidence or manual receipt import. The application engine must not endlessly retry OAuth and consume all worker capacity. Track message IDs and processed revisions so repeated polling is idempotent. Use a bounded mailbox cursor and backfill window.

Acceptance gates

1. OAuth refresh and revocation produce correct state transitions without exposing tokens.
2. Synthetic unrelated messages never trigger actions or appear in application records.
3. Account-created and application-received messages are correctly distinguished.
4. Two roles at the same employer do not receive the same receipt unless identity evidence supports it.
5. Unexpected verification-link domains are rejected.
6. Duplicate email delivery creates one outcome event.
7. The app remains useful with Gmail disconnected.

Evidence and handoff

Deliver sanitized email fixtures, a correlation confusion matrix, OAuth setup/runbook, and live validation status. If public OAuth verification is not available, label the supported personal deployment accurately and keep hosted public-distribution claims pending. P12 can use outcome and exception data for optimization but must not send entire emails to a general-purpose model.

P12. Bounded multi-agent orchestration and cost optimization

Outcome: agents improve preparation quality and efficiency under a measured request budget, without gaining authority over external submissions.

Depends on: P11. The simple working pipeline remains the reference implementation for comparison.

Work packages

- **P12-01: Graph contracts.** Implement extractor, match analyst, drafter, form-answer planner, critic, and bounded repair nodes with typed input/output and capability restrictions.
- **P12-02: Checkpointing.** Save calculation checkpoints with graph/prompt/schema versions. Resume compatible work; explicitly migrate or restart incompatible pure preparation.
- **P12-03: Batching.** Batch only independent similarly sized tasks, require item IDs, validate each result separately, and split on truncation or context pressure.
- **P12-04: Dynamic routing.** Select eligible free models by measured task quality, context needs, health, and latency. Keep model pinning subject to the free-only firewall.
- **P12-05: Evaluation and ablation.** Compare deterministic/template baseline, single-model generation, and drafter-plus-critic on the same held-out jobs. Measure claim errors, correction rate, latency, and requests per accepted packet.

Agent design constraints

No agent receives raw browser credentials or an unrestricted submit tool. A critic's acceptance is advisory input to deterministic gates, not authorization. Repair loops have a fixed maximum and do not create new facts. Failure to reach an accepted packet creates a useful exception with the unsupported claim or unanswered question.

Use cheap code for arithmetic, date logic, duplicate checking, known field mappings, and budget enforcement. Use models where language interpretation adds value. Do not add an agent for each database operation. Additional agents earn their place only when ablation tests show a measurable benefit over the simpler baseline.

Budget governor

Implement a daily allocation planner that reserves capacity for high-value packets and repairs. Give near-complete eligible applications priority over speculative low-fit research. Respect fairness so one long or failing job cannot consume the entire budget. Catalogue checks and health metadata should not become frequent inference probes that exhaust a restrictive account.

When a batch fails, retain valid items and retry only invalid ones where the response schema permits reliable separation. Guard against another job's company, salary, or facts entering an item. If an available free model cannot reliably produce the required format, disable it for that task rather than endlessly repairing output.

Acceptance gates

1. Agent execution has bounded steps, time, tokens, and request count.
2. Graph replay cannot perform or repeat a final submission.
3. Batch contamination, duplicate IDs, missing items, and truncated output are detected.
4. Free-only routing remains enforced under fallback and manual model selection.
5. The measured multi-agent variant improves a declared metric or is not enabled by default.
6. A 50-request simulation exhausts predictably, preserves reserve accounting, and defers work without falsifying success.

Evidence and handoff

Publish the sanitized evaluation method and actual result tables. Store prompt versions and regression cases. The public project can legitimately demonstrate agent orchestration here because it has bounded contracts and measurable benefits, not simply multiple role names. P13 tunes unattended throughput around the measured preparation and browser bottlenecks.

P13. Unattended Windows autopilot and throughput

Outcome: the local application runs independently of Codex, resumes across restarts, and works toward 50 appropriate confirmed applications per day with transparent bottlenecks.

Depends on: P12. Local execution requires the computer to remain powered, awake, and connected.

Work packages

- **P13-01: Background runtime.** Package a launcher and owner-approved startup mechanism. Add graceful shutdown, crash restart, health checks, and configuration migration.
- **P13-02: Scheduling.** Implement source-specific polling, queue priorities, daily targets, browser concurrency limits, quiet hours for notifications, and Europe/Amsterdam day boundaries.
- **P13-03: Operations dashboard.** Show actual stage counts, confirmed evidence, unresolved ambiguity, needs-help queue, next work, model quota estimate, and stop controls.
- **P13-04: Capacity testing.** Run synthetic day workloads with varied fit, model latency, account friction, portal errors, and challenges. Measure bottlenecks and memory growth.
- **P13-05: Controlled live ramp.** Under active policy, progress from the verified canary to small batches and then toward the requested target. Stop or isolate affected routes on correctness regressions, not merely on slow performance.

Scheduling details

Prioritize already prepared, high-fit, still-open jobs. Keep a small reserve for urgent newly discovered matches and reconciliation. Limit per-employer concurrent browser actions, and observe portal cooldowns. Do not defeat block responses to improve throughput. A blocked source should not repeatedly wake the whole system every few seconds.

Daily totals count confirmed application identities with receipt timestamps, not attempts. A submission initiated before midnight and confirmed after midnight is reported consistently with both timestamps available. Historical imports and mock submissions are separate counters. If fewer than 50 are possible, report the actual reason distribution.

Acceptance gates

1. Closing Codex does not stop the application runtime.
2. Restarting Windows or the worker resumes non-ambiguous work and reconciles in-flight attempts.
3. Sleep/wake and network-loss tests do not cause duplicate commits.
4. A 24-hour synthetic soak shows no unbounded queue, memory, disk, or retry growth.
5. Pause and emergency stop behave as documented, including the limitation for already-sent requests.
6. The dashboard can explain the target shortfall using measured counts.
7. Live ramp evidence is genuine; reaching 50 is not a release claim unless actually observed.

Evidence and handoff

Deliver install/start/stop instructions, synthetic day report, live ramp status, worker resource measurements, and a daily summary example. A local release can now be useful while later deployment work continues. P14 must preserve these behaviors in server and hybrid profiles rather than build a separate less-safe code path.

P14. Fully deployable server and hybrid profiles

Outcome: the same product runs on Windows, wholly on a server, or with server coordination and a local browser worker, with explicit operational differences.

Depends on: P13. Deployment to billable infrastructure requires the owner's configured provider/account and budget; creating those charges is not inferred from a request to write deployable software.

Work packages

- **P14-01: Server images and Compose.** Build reproducible images with pinned browser/runtime compatibility, PostgreSQL, persistent artifacts, authenticated API, TLS configuration, health checks, and restart policies.
- **P14-02: Secrets and storage.** Support environment-mounted or external secret-store references without baking credentials into images. Add S3-compatible storage contract tests and encryption configuration.
- **P14-03: Hybrid transport.** Implement outbound worker registration, authenticated command delivery, owner binding, command expiry, replay protection, lease renewal, and receipt acknowledgment.
- **P14-04: Secure browser handoff.** Provide owner-authenticated, expiring access to a single waiting server browser session using a reviewed private transport. Never expose raw CDP/debug endpoints publicly.
- **P14-05: Migration.** Export/import candidate and application state consistently, drain active commits, reconcile uncertainty, move artifact manifests, and require session reauthentication when necessary.

Deployment documentation

Provide minimum measured resource needs, storage estimates, operating-system assumptions, required network egress, ports, backup location, update procedure, and troubleshooting. Avoid naming a free hosting tier as a permanent guarantee. A server profile must include browser execution, not just host the dashboard while quietly depending on the owner's PC.

Use the same domain services and repository contracts across profiles. Local-only features should be behind explicit interfaces. Server authentication must protect every artifact download and session action; random object IDs are not authorization. Hybrid mode must reject stale commands after disconnection and reconcile any attempt that was already in flight.

Acceptance gates

1. A clean server deployment completes the full mock application workflow with the owner's PC disconnected.
2. Hybrid mode completes the same workflow with a local browser worker and server-owned state.
3. Replayed, expired, wrong-owner, and wrong-intent commands are rejected.
4. Loss of server connectivity prevents new local commits after lease/authorization expiry.
5. Database and browser-control ports are not publicly exposed in the production profile.
6. Local-to-server migration preserves application identity, packets, policy versions, and receipts.
7. TLS/authentication, backup, and resource requirements are documented and tested.

Evidence and handoff

Deliver deployment manifests, clean-install recordings, parity test reports, and a profile-comparison guide. Mark a real hosting deployment pending if credentials or paid infrastructure are absent, while still completing containerized verification locally. P15 tests the entire system against failures rather than treating a successful container startup as production readiness.

P15. Security, fault injection, restore, and release gates

Outcome: production claims are supported by adversarial, recovery, and operational tests, with known residual limitations documented.

Depends on: P14. Security and tests already exist from earlier phases; this phase adds integrated verification and remediation.

Work packages

- **P15-01: Threat review.** Test malicious job descriptions, email instructions, document payloads, unsafe imported URLs, untrusted model output, and attempts to widen execution authority.
- **P15-02: Crash matrix.** Execute every commit boundary under process kill, network loss, worker duplication, lease expiry, queue redelivery, and receipt-write failure.
- **P15-03: Recovery drills.** Back up, restore, rekey where supported, rotate credentials, revoke authorization, and simulate restoring a database older than external applications.
- **P15-04: Supply-chain checks.** Audit dependencies, pin CI actions where practical, generate an SBOM, scan release artifacts for secrets, and document provenance of copied code.
- **P15-05: Soak and performance.** Run 72-hour synthetic operation with outages and quota resets. Measure resource bounds, reconciliation lag, queue fairness, and parser drift response.

Required adversarial cases

A JD instructs the agent to email credentials; a PDF embeds an unrelated URL; an imported job link redirects to a private network address; a model returns another candidate's fields; a fake receipt appears before submission; a screenshot includes a password field; an attacker changes the intent's artifact path; and an expired worker tries to commit. The tests should assert rejected capability or quarantined data, not merely that a warning was logged.

Acceptance gates

1. No critical unresolved finding can permit unauthorized submission, secret disclosure, wrong-owner access, or blind duplicate replay.
2. Zero duplicate external records occur in the supported mock crash suite; unresolved outcomes are retained honestly.
3. All free-only adversarial pricing/fallback tests pass.
4. A restored system remains commit-disabled until the external-submission gap is reconciled.
5. Backup restoration verifies database consistency and artifact checksums.
6. The 72-hour synthetic soak completes with bounded resource usage and no silently lost tasks.
7. Real adapter claims are limited to the tested variants and dates.

Residual-risk reporting

Record portal-dependent uncertainties, manual challenge requirements, free-provider availability limits, and any external prerequisites still pending. Do not hide them inside an all-green dashboard. Distinguish a critical correctness failure from an unavailable optional integration. Remediate failures and rerun the affected gates; do not mark the phase complete by documenting a serious known defect as acceptable.

Evidence and handoff

Deliver the threat model, test matrix, fault report, restore report, SBOM, sanitized diagnostics bundle, and release-gate checklist. P16 can now measure prioritization and outcome quality without optimizing a system that miscounts submissions or loses evidence.

P16. Outcomes, calibration, and continuous improvement

Outcome: the product learns which sources and targeting choices produce useful outcomes, while keeping factual constraints and sample-size honesty.

Depends on: P15. Real interview/outcome analysis is naturally delayed; synthetic analytics tests can be completed before enough real data accumulates.

Work packages

- **P16-01: Outcome ledger.** Track receipt, screening, interview, rejection, offer, owner withdrawal, and unknown outcome with source and date. Keep recruiter messages and manual corrections auditable.
- **P16-02: Cohort reporting.** Compare sources, role families, seniority, sponsorship signals, and document variants using fixed application cohorts and suitable observation windows.
- **P16-03: Score calibration.** Evaluate whether high fit scores correspond to better reviewed relevance and eventually better outcomes. Do not publish interview probabilities before enough reliable labeled outcomes exist.
- **P16-04: Quality regression loop.** Turn corrected facts, rejected claims, misclassified forms, and adapter drift into sanitized regression fixtures.
- **P16-05: Controlled experiments.** Compare approved wording or ordering variants without applying twice to the same job or weakening truthfulness. Record experiment assignments before outcomes are known.

Analysis rules

No response is not automatically a rejection. Recent applications have had less time to receive an outcome, so report cohort age and unresolved counts. A high interview rate from five applications is not a reliable broad claim. Segment visa/language rejection reasons only when the employer or owner actually provides them; do not infer sensitive causal explanations from silence.

Measure source yield as appropriate confirmed applications and later outcomes per unit effort. An adapter with many failures may be a better engineering investment than adding another low-yield source. The system can suggest expanding approved adjacent roles, but it must not silently change the owner's scope, sponsorship answers, or salary policy.

Acceptance gates

1. Every outcome is linked to an existing application with provenance.
2. Recent and unresolved applications are not mislabeled as rejections.
3. Reported conversion rates include numerator, denominator, date range, and relevant exclusions.
4. No UI uses match score as a fabricated selection probability.
5. Experiments cannot create duplicate applications or unsupported claims.
6. Owner corrections become regression tests without leaking personal data into Git.
7. A recommendation to change policy requires a new recorded owner policy revision before it affects commits.

Evidence and handoff

Deliver analytics tests, sample synthetic dashboards, calibration methodology, and any genuine preliminary results clearly labeled with sample size. If real outcome data is insufficient, say so and leave longitudinal evaluation ongoing without pretending the coding phase is blocked. P17 publishes the verified product and its measured limitations.

P17. Release, portfolio, and autonomous maintenance handoff

Outcome: a reproducible, installable, documented release with a convincing public demonstration and an honest support matrix.

Depends on: P16. All mandatory correctness gates must pass; missing optional external integrations are explicitly labeled.

Work packages

- **P17-01: Release package.** Produce versioned Windows and server artifacts, checksums, release notes, migration instructions, and a rollback plan. Rollback must preserve the application ledger or disable commits until reconciled.
- **P17-02: Public repository.** Finalize README, architecture, ADRs, setup guides, contribution/security policies, license, upstream attribution, issue templates, and adapter status.
- **P17-03: Demonstration.** Create a synthetic end-to-end demo showing fit explanations, truthful documents, actual mock submission, receipt evidence, crash recovery, and challenge handoff.
- **P17-04: Clean-room verification.** Follow the published instructions on a clean environment and confirm demo, tests, build, deployment, and backup/restore work without hidden local files.
- **P17-05: Maintenance plan.** Define dependency updates, source/adapter drift review, model-catalogue refresh, immigration-rule refresh, incident handling, and recurring regression checks.

README narrative

Lead with the problem, what the system actually automates, and a short demo. Explain the deterministic commit boundary and why agents are bounded. Show a small architecture diagram, installation path, support matrix, privacy/cost settings, and measured reliability. Separate implemented features, experimental adapters, and roadmap. Link full evaluation methodology instead of placing unsupported superlatives above a screenshot.

Public screenshots and recordings use fictional names, synthetic jobs, and mock receipts. No real applications are triggered by CI or the public demo. If genuine submission totals are published later, obtain the owner's choice about what to disclose and anonymize employer/candidate details appropriately.

Final acceptance gates

1. A new user can install and run the synthetic demo from published instructions.
2. The private installation can make authorized real final submissions on live-verified adapters.
3. Local, server, and hybrid deployment behavior is documented and verified at the claimed level.
4. All R01-R12 requirements have a final evidence entry or a clearly scoped external verification status.
5. No critical security/correctness issue remains open.
6. The repository and release artifacts contain no real credentials, personal CVs, browser sessions, or private receipts.
7. The release notes distinguish actual benchmark results from targets and synthetic demonstrations.

Final handoff

Provide the owner with current deployment status, setup items still requiring them, active submission policy, supported portals, latest backup location, restore instructions, stop controls, current quota constraints, and next maintenance tasks. Summarize genuine confirmed submissions separately from prepared packets. The software should now run without Codex; future coding sessions use the saved ledger, tests, and runbooks to maintain it.

Do not declare success because every phase heading has a checkbox. Success is working software, passed gates, reproducible deployment, and evidence of actual authorized submission on the supported live routes.

Appendix A. Interface contracts and configuration examples

Application API surface

These are proposed application-owned endpoints, not employer API endpoints. Publish an OpenAPI contract and generate request validation from the same schemas used by workers. All state-changing endpoints authenticate the owner, enforce object ownership, support optimistic concurrency where relevant, and return structured error codes. Pagination uses stable cursors.

Endpoint	Purpose and constraints
GET /v1/jobs	Filter by eligibility, source, freshness, portal, score, and status
POST /v1/jobs/import	Validate URL/source and enqueue discovery; never submit directly
GET /v1/jobs/{id}/assessment	Explain gates, evidence, gaps, and score components
POST /v1/profiles/import	Create a source ingestion task and immutable source hash
POST /v1/profiles/{id}/publish	Publish reviewed facts as a new version
GET /v1/authorizations/active	Return plain-language scope plus machine policy
POST /v1/authorizations	Create an authenticated standing authorization revision
POST /v1/authorizations/{id}/revoke	Prevent future commits and invalidate readiness
POST /v1/applications/{id}/prepare	Enqueue preparation with a logical idempotency key
POST /v1/applications/{id}/enqueue	Request processing through gates, never bypass them
GET /v1/applications/{id}/packet	Inspect exact documents, answers, versions, and QA
GET /v1/applications/{id}/evidence	Retrieve owner-authorized receipt evidence
POST /v1/exceptions/{id}/resolve	Record an answer/action and revalidate affected work
POST /v1/control/pause	Pause selected stages with actor and reason
POST /v1/control/stop	Revoke new execution capability and trigger cancellation
GET /v1/operations/summary	Stage counts, target gap, health, quota estimate

Do not expose a public endpoint that accepts an arbitrary URL and arbitrary browser script with commit authority. Internal browser commands are typed, owner-bound, application-bound, expiring, and tied to a stored intent. A transport idempotency key prevents duplicate command acceptance; it is not a guarantee that the external portal supports idempotency.

Proposed policy shape

```
{
  "schemaVersion": 1,
  "ownerId": "synthetic-owner",
  "mode": "auto_submit",
  "environment": "private_local",
  "countries": ["NL"],
  "dailyConfirmedTarget": 50,
  "dailyAttemptCap": 65,
  "allowAccountCreation": true,
  "allowPaidInference": false,
  "allowPaidServices": false,
  "requireSupportedAdapter": true,
  "requireReceiptEvidence": true,
  "unknownRequiredAnswer": "needs_input",
  "captchaAction": "handoff_and_continue",
  "duplicatePolicy": "block_same_requisition",
  "profileVersion": "profile-reviewed-001",
  "approvedAnswerSet": "answers-reviewed-001",
  "effectiveAt": "owner-configured-timestamp",
  "expiresAt": "owner-configured-timestamp"
}
```

This is a schema illustration, not ready-to-load production authorization. The timestamp placeholders must fail validation. The daily attempt cap is separately configurable because unsuccessful attempts and confirmed applications are different counters. The public demo defaults to external submission disabled. A private owner can explicitly enable the illustrated auto-submit policy after onboarding.

Error vocabulary

Use stable codes such as JOB_CLOSED, DUPLICATE_CONFIRMED, DUPLICATE_SUSPECTED, PROFILE_STALE, POLICY_REVOKED, ANSWER_UNKNOWN, FORM_CHANGED, UPLOAD_FAILED, CHALLENGE_REQUIRED, SESSION_EXPIRED, ADAPTER_UNSUPPORTED, MODEL_QUOTA_EXHAUSTED, MODEL_ROUTE_INELIGIBLE, CLAIM_UNSUPPORTED, LEASE_STALE, COMMIT_UNKNOWN, RECEIPT_UNCORRELATED, and STORAGE_UNAVAILABLE. Each error has retryability, stage, owner action if any, redacted details, and a correlation ID.

Event envelope

```
event_id, schema_version, occurred_at
owner_id, aggregate_type, aggregate_id, aggregate_revision
event_type, actor_type, actor_id
correlation_id, causation_id, redacted_payload
```

Use events for observability and downstream work, not as a second independent authority over application state. Consumers must tolerate redelivery. A consumer cannot infer authorization from receiving an event; it checks the current policy at the relevant boundary.

Appendix B. Test matrix and definition of done

Minimum integrated scenarios

Test ID	Scenario	Expected invariant
T01	Same job appears on three sources	One application identity, all sources retained
T02	Same title, distinct requisitions	No unjustified merge
T03	Historical letter exists without receipt	Provenance preserved; not newly confirmed
T04	Explicit incompatible Dutch requirement	Hard gate before expensive preparation
T05	Sponsorship unknown, otherwise suitable	Uncertainty retained, not invented rejection/approval
T06	Salary range crosses route threshold	Conditional feasibility, correct units
T07	Overlapping work periods	No inflated years of experience
T08	In-development project	Cannot be rewritten as deployed
T09	Model returns invented metric	Packet rejected even with valid JSON
T10	Pinned model becomes paid	Request rejected before inference
T11	Provider privacy filters yield no route	Pause inference without relaxing privacy
T12	Batch uses another company's name	Affected packet rejected
T13	Required field appears after upload	Reinspection catches it
T14	File selection succeeds, upload fails	Not READY
T15	Fake success banner before submission	Not CONFIRMED
T16	Crash before final click, after in-flight	Reconciliation, no blind replay
T17	Server accepts, response lost	UNKNOWN then receipt reconciliation
T18	Two workers race one application	One valid committing owner
T19	Authorization revoked before click	No external application
T20	Stop after request already sent	Actual outcome retained; no false undo claim
T21	CAPTCHA interrupts one job	Other jobs continue
T22	Owner manually submits during handoff	Detect/reconcile, do not submit again
T23	Account-created email arrives	Not an application receipt
T24	Verification link redirects unexpectedly	Link rejected
T25	Worker clock/day boundary changes	Stable UTC records and correct local counters
T26	Restore backup older than submissions	Commit disabled until gap reconciled

Test ID	Scenario	Expected invariant
T27	Hybrid command replay after expiry	Reject stale command
T28	Malicious JD asks for secrets	No authority or secret disclosure
T29	Job URL targets private network	Fetch rejected
T30	Model budget exhausted mid-day	Prepared/cached work continues honestly
T31	Parser returns empty feed after drift	No mass false closure
T32	Browser session token expires	Account exception, no unbounded retry

Test layers

Unit tests cover pure calculations, schema validation, state transitions, and exact policy meaning. Property-based tests cover date overlaps, URL normalization, idempotency keys, state-machine sequences, and arbitrary malformed model outputs. Repository contract tests run on both database backends. Adapter contract tests use the mock ATS. End-to-end tests exercise UI, API, queue, documents, browser, and receipts together.

Fault injection is a separate suite, not a few happy-path retries. Tests must observe the mock employer's server-side application count. Security tests assert denied actions and protected data. Visual document tests check representative PDFs; accessibility tests cover keyboard navigation and understandable status/error messages. Performance tests measure distributions and resource use rather than a single best run.

Per-ticket definition of done

The implementation exists and is reachable; acceptance tests pass; regression tests cover the bug or behavior; errors and cancellation are handled; logs are redacted; documentation and schemas are updated; evidence paths are recorded; and no TODO masquerades as a production feature. A fixture-based success is labeled fixture-tested. Live verification requires live evidence. A missing external credential is recorded as a prerequisite, not replaced with fabricated output.

Release evidence bundle

Include commit/version, runtime versions, dependency lock hashes, requirement traceability, test reports, evaluation dataset version, adapter support matrix, synthetic screenshots, crash/restore evidence, deployment parity results, and known limitations. Private live receipts are stored separately and referenced by opaque IDs in public reports. The public bundle must be safe to upload without manual removal of secrets after every run.

Appendix C. Operator runbooks

Runbook 1: first private launch

1. Install the tested release and run diagnostics. Select a non-synced operational data directory.
2. Import the original CV and historical applications. Review only material factual conflicts and missing answers.
3. Configure Netherlands scope, availability, sponsorship wording, salary answers, and account-creation preferences.
4. Connect OpenRouter with free-only mode and inspect the actual available budget/routes. Optional Gmail setup can wait.
5. Launch the owned browser and sign into necessary accounts. Keep personal browser profiles separate unless explicitly supported.
6. Run the synthetic demo and inspect one representative generated packet.
7. Enable standing auto-submit authorization for the intended scope. Process a genuine eligible canary and verify its receipt.
8. Activate the requested daily target, monitor the ramp, and keep the exception queue available without requiring a click for every routine application.

Runbook 2: an application is stuck after Submit

Do not press Submit again. Open the attempt record, confirm whether IN_FLIGHT was persisted, inspect the correlated portal state, then check account history or receipt email. If receipt evidence is found, attach it and confirm the existing application. If a definitive rejection of the form is established, repair the supported issue and create a new intent. If evidence remains ambiguous, leave it for review while other applications proceed.

Runbook 3: free models stop responding

Check route health, account budget state, recent 429/5xx errors, and provider eligibility. Honor cooldowns. Try only another already eligible free route within budget. Continue discovery, deduplication, cached packets, deterministic answers, and reconciliation. Do not enable paid fallback, create new accounts, or relax privacy to make a status indicator green. Report the actual number of model-dependent jobs deferred.

Runbook 4: portal layout changes

Pause that adapter/version, not the whole system. Save a sanitized failure snapshot and identify the changed contract. Add a failing fixture, implement the correction, run the common and variant-specific suites, and validate a non-submitting live inspection. Re-enable commits only at the tested support level. Revalidate queued forms created under the old version.

Runbook 5: secret exposure or suspicious behavior

Stop new commits, revoke affected tokens/sessions, and preserve minimal incident evidence without copying secrets into tickets. Determine the affected owner, routes, and time interval. Rotate credentials, invalidate browser sessions where needed, inspect application attempts for unexpected destinations, fix the root cause, and rerun the relevant gates. Resume only after the active policy and credentials are known-good.

Runbook 6: backup and restore

Pause new commits and let safe preparation finish. Snapshot the database consistently, store manifests and artifacts, encrypt the backup, and verify checksums. For restoration, use a clean validated location, restore compatible schema/configuration, and keep commits disabled. Identify applications potentially sent after the backup and reconcile them against receipts. Only then reauthorize normal processing. Never fix a restore conflict by deleting the application ledger.

Runbook 7: moving from Windows to a server

Drain active commits and record all unknown attempts. Export the profile, policy history, database, and artifact manifest consistently. Import into the server profile and verify owner scoping and checksums. Reauthenticate sessions where platform-specific encryption prevents safe transfer. Test mock submission, TLS, backups, and challenge handoff. Reconcile uncertainty before enabling server commits, and disable the old local committing authority.

Runbook 8: the system sends too few applications

Inspect the funnel in order: source supply, unique live vacancies, eligibility, model capacity, packet acceptance, adapter support, account readiness, and receipt verification. Fix the largest measured bottleneck. Add high-yield sources or an adapter when coverage is the issue; improve evidence mapping when genuinely suitable roles are misclassified. Do not weaken factual answers, repeat applications, or rename attempts as successes to reach 50.

Appendix D. Autonomous coding execution protocol

Instruction to the implementing coding agent

Treat this specification as the implementation contract. Inspect the repository first. Continue phase by phase through working code, tests, documentation, and evidence. Do not stop after scaffolding, mockups, a planning document, or a tracker. The central requirement is authorized final application submission with receipt verification. Keep the design local-first and fully deployable, with free-only OpenRouter inference and bounded agents.

At the start of every resumed session, read repository instructions, the architecture decisions, the phase ledger, the most recent handoff, and Git status. Determine what is actually complete from code and evidence. Preserve unrelated user work. Select the earliest unblocked incomplete ticket whose dependencies passed. Keep changes focused and run the relevant tests before advancing.

Ledger format

```
{
  "phase": "P08",
  "ticket": "P08-03",
  "status": "in_progress",
  "dependsOn": ["P08-01", "P08-02"],
  "acceptanceTests": ["T15", "T17"],
  "evidence": [],
  "externalPrerequisites": [],
  "lastVerifiedCommit": null,
  "nextAction": "Implement correlated receipt classification"
}
```

Allowed statuses are not_started, in_progress, code_complete_verification_pending, blocked_external, failed_gate, and complete. A phase is complete only when its mandatory gates pass. Keep optional live-validation prerequisites distinct so independent coding can proceed. Do not globally mark a phase complete because a mock passed when the contract also requires a genuine live receipt.

Execution loop

1. Read current state and identify the smallest useful ticket.
2. Implement production behavior with explicit failure handling.
3. Add or update tests, including a negative or recovery case.
4. Run targeted checks, then the appropriate broader regression suite.
5. Inspect actual results and fix failures; do not ignore red output.
6. Update docs, evidence, support labels, and the phase ledger.
7. Continue to the next unblocked ticket while within the authorized work.
8. Before any session ends, write a precise handoff so the next session can resume without guessing.

What to do when external setup is missing

Build and verify offline fixtures, adapters, configuration validation, and integration contracts. Record the exact missing item, such as an OAuth client, OpenRouter key, browser login, or eligible live vacancy. Continue independent work. Never fabricate a credential, live receipt, test result, or successful deployment. Do not start billable infrastructure or change external permissions merely to remove a blocker. The owner can supply setup once and the next session can resume the pending validation.

Handoff content

Record current branch/worktree, files changed, phase/ticket status, tests run and actual results, evidence locations, known failures, external prerequisites, next command or concrete action, and any active processes. Do not include secrets. Distinguish proposed work from implemented work. A future session should be able to recover intent from repository files, not rely on an agent's memory.

Optional parallel engineering

If the owner authorizes parallel coding agents, assign bounded independent packages with explicit ownership and contracts. Good splits include fixture creation, document QA, and a separately specified connector. Do not let multiple agents edit the commit state machine or migration schema simultaneously without one integration owner. Integrate through tests; an agent's claim of completion is not evidence. Do not use additional agents simply to increase activity.

Phase completion report

Use a short report: outcome delivered; requirements satisfied; tests and evidence; live-versus-mock status; remaining external setup; next phase. Do not repeatedly ask whether to continue when the owner has already requested the full build. Persist progress and resume across sessions; no single chat session is assumed to run indefinitely.

Appendix E. Priorities, tradeoffs, and future extensions

What to optimize first

The first priority is correctness of candidate data and application identity. The second is one complete submission route with evidence. The third is high-yield portal coverage. The fourth is unattended reliability and free-budget efficiency. Deployment, advanced agents, and portfolio polish then extend a working product. This sequence avoids spending most of the project on a sophisticated research system that never sends an application.

Keep the initial architecture small enough for one developer to operate. A single owner with two browser workers does not need enterprise-scale orchestration infrastructure by default. Use interfaces and parity tests to retain a path to scale. Optimize from measured bottlenecks, not speculation about future millions of users.

Future extensions after P17

- Add complex account-heavy ATS families when actual candidate coverage justifies them.
- Add authorized recruiter-outreach or calendar workflows only under a new explicit scope, with separate message/booking controls.
- Add local model inference as an optional fallback if hardware and latency become acceptable; do not make it required for the free-cloud-model design.
- Add a public multi-user service only after tenant isolation, abuse controls, OAuth verification, privacy obligations, billing, and operational ownership are separately specified.
- Consider a different workflow engine only when measured scheduling or operational requirements exceed the database-backed design; preserve commit semantics during migration.
- Add more sophisticated semantic retrieval if the evidence corpus grows enough to justify it. A small structured fact ledger does not automatically need a vector database.

Honest limitations

No software can guarantee interviews, unlimited eligible vacancies, sponsorship, acceptance of an EOR arrangement, unchanging portal forms, or unlimited free inference. Those are external constraints. The product can nevertheless provide substantial value by broadening relevant discovery, reducing repeated manual form work, improving factual tailoring, promptly submitting supported applications, and making every success or blocker visible.

The owner remains in control of facts, preferences, authorization, and exceptional personal declarations. That does not require the owner to click Submit for each ordinary application. The distinction is central: autonomous operation within a clear policy, with narrow exceptions and evidence-backed final submissions.

Appendix F. Sources and verification notes

Sources were checked on 16 September 2026. Product/API behavior, model availability, quotas, immigration rules, and upstream code can change. Recheck them at implementation and before release. The technical architecture, phase plan, tests, and acceptance targets in this document are proposed original design decisions unless a source is cited. They are not claims that the project already exists or has passed these tests.

S01. Career Ops repository and license. [Pinned reference repository](#). Supports the reference identity, repository inspection, and attribution review.

S02. Career Ops architecture. [Pinned architecture documentation](#). Reference workflow and architectural context; not proof of autonomous final submission.

S03. Career Ops scanner. [Pinned scan implementation](#). Connector, discovery, normalization, and deduplication design inspected.

S04. Career Ops OpenRouter runner. [Pinned runner implementation](#). Free-model selection, failure handling, and answer-preparation behavior inspected.

S05. Career Ops application preparation. [Pinned preparation implementation](#). Preparation output is distinct from verified submission.

S06. Career Ops artifact handling. [Pinned artifact module](#). Reviewed packet provenance and artifact-versioning patterns.

S07. Career Ops application answers. [Pinned answer module](#). Reviewed exact answer and selection snapshots.

S08. Greenhouse Job Board API. [Official documentation](#). Public listing reads and authenticated employer-side application submission are different capabilities.

S09. Lever Postings API. [Official repository/documentation](#). Public listings and application API-key requirements.

S10. Ashby public job posting API. [Official documentation](#). Public board listing interface; no assumed candidate submission authority.

S11. OpenRouter limits and free-tier guidance. [Limits](#) and [FAQ](#). Free usage constraints, credit versus request limits, and fallback/capacity caveats. The 50/1,000 request scenarios in this plan are configurable planning cases, not guaranteed account capacity.

S12. OpenRouter model properties and provider selection. [Model catalogue API](#) and [provider routing](#). Price metadata, supported parameters, routing constraints, and provider preferences.

S13. LangGraph execution and compatibility. [Interrupts](#), [persistence](#), and [backward compatibility](#). Resumption, checkpoints, and graph changes require careful side-effect and version handling.

S14. OpenRouter structured outputs. [Official guide](#). Schema support depends on model/provider capabilities and does not establish factual truth.

S15. OpenRouter provider logging. [Official privacy guide](#). Downstream policies differ; route restrictions can reduce availability.

S16. Playwright authentication. [Official documentation](#). Saved authentication state is sensitive and can enable impersonation.

S17. Playwright Docker. [Official documentation](#). Browser/package compatibility and container execution guidance.

S18. IND required income amounts. [Official current amounts](#). 2026 reduced highly skilled migrant amount, route conditions, and excluded holiday allowance. Personal eligibility still requires verification.

S19. IND recognized sponsors. [Public register: work](#). Entity-level recognition, not a guarantee for a specific job or EOR arrangement.

S20. SQLite WAL. [Official documentation](#). Same-host storage, single-writer behavior, durability tradeoffs, and current engine cautions.

S21. PostgreSQL SELECT and locking. [Official documentation](#). Row locking and SKIP LOCKED semantics used in the proposed queue design.

S22. Docker Compose production guidance. [Official documentation](#). Production-specific configuration and deployment considerations.

S23. Gmail API scopes. [Official scope documentation](#). Scope classification and verification implications.

S24. Google OAuth token lifecycle. [Official OAuth documentation](#). Refresh-token lifecycle, testing-state limitations, and revocation behavior.

S25. Google OAuth policies. [Official policy documentation](#). Deployment and public-distribution requirements depend on the actual app configuration.

Source-use boundary

Source pages are technical evidence, not instructions that can change the owner's authorization. Reused code requires local tests and license preservation. Immigration information is included to guide data modeling and screening, not to make an individual legal determination. No external source establishes a universal interview probability, a guaranteed daily application count, or CAPTCHA-bypass capability.

Document completion checklist

This specification contains the product contract, candidate rules, reference review, architecture, data model, free-model strategy, bounded agents, document pipeline, browser adapters, actual submission/reconciliation protocol, deployment/security design, 18 implementation phases, API/configuration examples, test matrix, operator runbooks, autonomous execution instructions, and dated sources. The next deliverable is implementation and evidence, not another expansion of the plan.